

常州工学院

CHANGZHOU INSTITUTE OF TECHNOLOGY

课程设计报告

课程名称: 专业方向课程设计

二级学院: 计算机信息工程学院

课题名称: 基于 RAG 与多智能体协作的
知识服务平台

班 级: 23 软一

学 号: 23030327

姓 名: 许子祺

指导教师: 蒋巍

2026 年 7 月

基于 RAG 与多智能体协作的知识服务平台

摘 要

现有知识管理系统普遍存在检索精度不足、智能体缺乏自主反思能力、 workflow 编排灵活性受限等问题。针对上述挑战，本文设计并实现了一个基于 RAG 与多智能体协作的知识服务平台。

平台采用“控制面 + 认知面”双核架构。控制面基于 Spring Boot 3 构建，负责权限管理、RAG 知识引擎与 DAG workflow 编排；认知面以 Hermes 内核为核心，通过 gRPC 实现反思循环推理与 AI Judge 三维质量评分。RAG 引擎支持文档语义切块、三路并行召回与 Reciprocal Rank Fusion 融合，并引入 CRAG 机制对检索结果自动修正。智能体编排支持 SupervisorAgent 任务分解、WorkerAgent ReAct 推理与三层记忆体系。workflow 引擎基于 DAG 拓扑排序实现多节点并行执行，支持条件网关与断点恢复。质量评估集成 AI Judge、RAGAS 与 RAGChecker，形成完整闭环。

前端采用 Vue 3 构建可视化 DAG 画布与对话界面，后端使用 PostgreSQL + PgVector 实现向量检索，Neo4j 支撑知识图谱，Redis 提供缓存管理。实验表明，平台在 RAG 检索任务上达到了较高的召回精度，多智能体协作机制有效提升了复杂问答质量。

关键词：RAG；多智能体；知识服务；workflow 编排；Hermes 认知内核

A Knowledge Service Platform Based on RAG and Multi-Agent Collaboration

Abstract

Existing knowledge management systems commonly suffer from low retrieval accuracy, lack of agent self-reflection, and limited workflow flexibility. To address these challenges, this paper designs and implements a knowledge service platform based on RAG and multi-agent collaboration.

The platform adopts a dual-core architecture of control plane and cognitive plane. The control plane, built on Spring Boot 3, handles authorization, the RAG knowledge engine, and DAG workflow orchestration. The cognitive plane, centered on the Hermes kernel, implements reflective reasoning and AI Judge quality scoring via gRPC. The RAG engine supports semantic chunking, tri-channel parallel retrieval with Reciprocal Rank Fusion, and a CRAG mechanism for automatic correction. Agent orchestration enables SupervisorAgent task decomposition, WorkerAgent ReAct reasoning, and a three-layer memory system. The workflow engine achieves parallel execution via DAG topological sorting with conditional gateways and checkpoint recovery. Quality evaluation integrates AI Judge, RAGAS, and RAGChecker into a closed loop.

The frontend provides a visual DAG canvas and conversation interface using Vue 3. The backend employs PostgreSQL with PgVector for vector retrieval, Neo4j for knowledge graphs, and Redis for caching. Experiments show that the platform achieves high recall precision, and multi-agent collaboration effectively improves complex QA quality.

Keywords: RAG; multi-agent; knowledge service; workflow orchestration; Hermes cognitive kernel

目 录

第 1 章 绪论	1
1.1 研究背景与意义	1
1.2 国内外研究现状	1
1.3 研究内容与主要贡献	2
第 2 章 系统设计与方法	3
2.1 系统总体架构	3
2.1.1 双核架构设计	3
2.1.2 模块依赖与防腐层设计	5
2.2 RAG 检索引擎设计	5
2.2.1 文档预处理：语义切块策略	6
2.2.2 查询转换与意图分析	8
2.2.3 四路并行召回	10
2.2.4 候选融合：Reciprocal Rank Fusion	13
2.2.5 两阶段重排序	13
2.2.6 Corrective RAG 质量评估	14
2.3 Hermes 认知内核设计	15
2.3.1 反思循环推理	15
2.3.2 多智能体协作	17
2.3.3 三层记忆体系	19
2.3.4 大语言模型工具调用的韧性工程设计	20
2.4 DAG workflow 引擎设计	22
2.4.1 拓扑排序与并行分组	22
2.4.2 执行引擎	23
2.4.3 双层持久化检查点	24
2.4.4 节点类型体系与模板方法设计	24
2.4.5 数据流图与全局上下文管理	25
2.4.6 表达式引擎安全设计	26
2.5 质量评估框架设计	26
2.5.1 AI Judge 三维在线评分机制	26
2.5.2 RAGAS 离线评估	27

2.5.3	RAGChecker 声明级蕴含判断.....	27
2.5.4	LangFuse 遥测与全链路可观测性 (Observability)	28
第 3 章	系统具体实现	29
3.1	开发环境与技术选型	29
3.2	gRPC 通信协议与底层网络实现	29
3.3	RAG 检索引擎实现	30
3.3.1	文档切块实现	30
3.3.2	向量存储实现	31
3.3.3	检索服务实现	31
3.3.4	上下文构建实现	32
3.4	Hermes 认知内核实现	32
3.4.1	反思循环实现	32
3.4.2	智能体编排实现	32
3.4.3	Plan-and-Solve 实现	33
3.4.4	记忆系统实现	34
3.5	DAG workflow 引擎实现	34
3.5.1	节点执行模板	34
3.5.2	条件分支实现	34
3.5.3	流式输出实现	34
3.5.4	Human-in-the-loop 人工审批与挂起恢复实现	35
3.6	前端实现	35
3.6.1	DAG 画布与前端工程化实现	35
3.6.2	流式通信实现	36
3.7	安全实现	36
3.7.1	无状态 JWT 认证与防重放实现	36
3.7.2	RBAC 授权	36
第 4 章	实验	37
4.1	实验环境	37
4.2	功能验证	37
4.2.1	RAG 知识引擎	37
4.2.2	Hermes 认知内核	38
4.2.3	DAG workflow 引擎	39

4.3	性能基准测试	40
4.4	质量评估	40
4.4.1	RAGAS 七维评估	40
4.4.2	RAGChecker 声明级蕴含判断	41
4.5	本章小结	41
第 5 章	总结与展望	43
5.1	工作总结	43
5.2	不足与展望	43
5.2.1	当前不足	43
5.2.2	未来展望	43
致 谢		45
参考文献		47

第 1 章 绪论

1.1 研究背景与意义

随着大语言模型 (LLM) 技术的快速发展,人工智能在自然语言处理、知识推理和代码生成等领域取得了突破性进展,企业对智能化知识管理与自动化业务流程的需求日益迫切。然而,现有知识管理系统在实际应用中仍面临诸多挑战:传统的基于关键词的检索方法难以理解用户查询的语义意图,单一检索路径在面对复杂查询时召回率有限;现有 LLM 智能体系统通常采用“生成即返回”的模式,缺少对生成结果的质量评估和自修正机制,容易产生幻觉回答;工作流引擎多采用线性结构,缺乏并行执行与断点恢复能力;企业知识分散在文档、数据库、知识图谱等多种载体中,缺乏统一的融合检索机制^[1]。针对上述问题,本文设计并实现了一个基于 RAG (Retrieval-Augmented Generation)^[2]与多智能体协作的知识服务平台。该平台采用“控制面+认知面”双核架构,控制面基于 Spring Boot 3 构建,负责权限管理、RAG 知识引擎与 DAG 工作流编排;认知面以自研 Hermes 内核为核心,通过 gRPC 协议与控制面通信,实现反思循环推理与质量评分。在检索层面,平台通过“四路并行召回+RRF 融合”策略和 CRAG^[3]质量评估机制显著提升了检索精度;在推理层面,通过“反思循环+自洽性检测”双重质量保障机制有效识别和修正幻觉回答,为企业级智能知识服务提供了完整的解决方案。

1.2 国内外研究现状

RAG 技术方面, Lewis 等人^[2]提出了 RAG 框架, Karpukhin 等人^[4]的密集检索显著优于 BM25, Cormack 等人^[5]的 RRF 实现了多路融合。在质量保障方面, CRAG^[3]引入检索质量评估, Self-RAG^[6]实现了自反思机制。重排方面, ColBERT^[7]的后期交互架构降低了计算开销。

多智能体协作方面, ReAct^[8]范式将推理与行动交替进行, CoT^[9]引导模型生成中间推理步骤, Plan-and-Solve^[10]实现了先规划后执行。MemGPT^[11]借鉴虚拟内存思想实现了分层记忆管理, LightRAG^[12]通过知识图谱双层检索提升了查询效果。

工作流与评估方面, Kahn^[13]的拓扑排序算法为 DAG 工作流的依赖解析提供了理论基础。在 AI 工作流编排中, LangGraph 等框架将智能体推理过程建模为有向图,支持条件分支、并行执行和持久化检查点。质量评估方面, RA-

GAS^[14]提出了七维度 RAG 评估体系，涵盖忠实度、相关性、精确度和召回率等指标；RAGChecker^[15]采用声明级蕴含判断方法，在 NeurIPS 2024 上被证明与人类评价具有更高的相关性。LangFuse 等可观测性平台则提供了从检索到生成的全链路追踪能力。

1.3 研究内容与主要贡献

(1) 提出”控制面+认知面”双核架构：控制面负责 RAG 引擎与 DAG workflow 编排，认知面以 Hermes 内核为核心实现反思循环推理，两者通过 gRPC 完全解耦。

(2) 设计四路并行召回+RRF 融合检索引擎：同时运行向量、关键词、元数据和知识图谱四条检索路径，通过 RRF 融合排序并引入 CRAG^[3]质量评估与自动修正。

(3) 提出带自洽性检测的反思循环机制：实现”生成→AI Judge 三维评分→Jaccard 自洽性检测→修正”的双重质量保障闭环。自洽性检测通过对同一提示词进行多次独立采样并计算成对 Jaccard 相似度，有效识别”看似合理但实际不可靠”的幻觉回答。

(4) 实现 DAG 并行 workflow 引擎：基于 Kahn 算法^[13]拓扑排序与 BFS 分层并行分组，支持 8 种节点类型（含条件网关、人工审批和知识检索），通过双层持久化检查点（Redis 快照级+PostgreSQL 分组级）实现断点恢复。

(5) 构建完整质量评估体系：集成 RAGAS^[14]七维评估、RAGChecker^[15]声明级蕴含判断和 LangFuse 全链路追踪。

第 2 章 系统设计与方法

本章详细阐述平台的整体架构设计和核心算法原理。首先介绍系统的双核架构和模块划分，然后依次论述 RAG 检索引擎^[2]、Hermes 认知内核^[6]、DAG 工作流引擎^[13]和质量评估框架^[14-15]的设计与方法。每个算法均给出形式化定义、数学推导和关键参数分析。本章引用的核心文献涵盖检索增强生成^[1-2]、密集向量检索^[4,16]、排序融合^[5,7]、自反思推理^[3,6]、多智能体协作^[8-10]、分层记忆管理^[11]、图增强检索^[12]以及质量评估^[14-15]等前沿研究成果。

2.1 系统总体架构

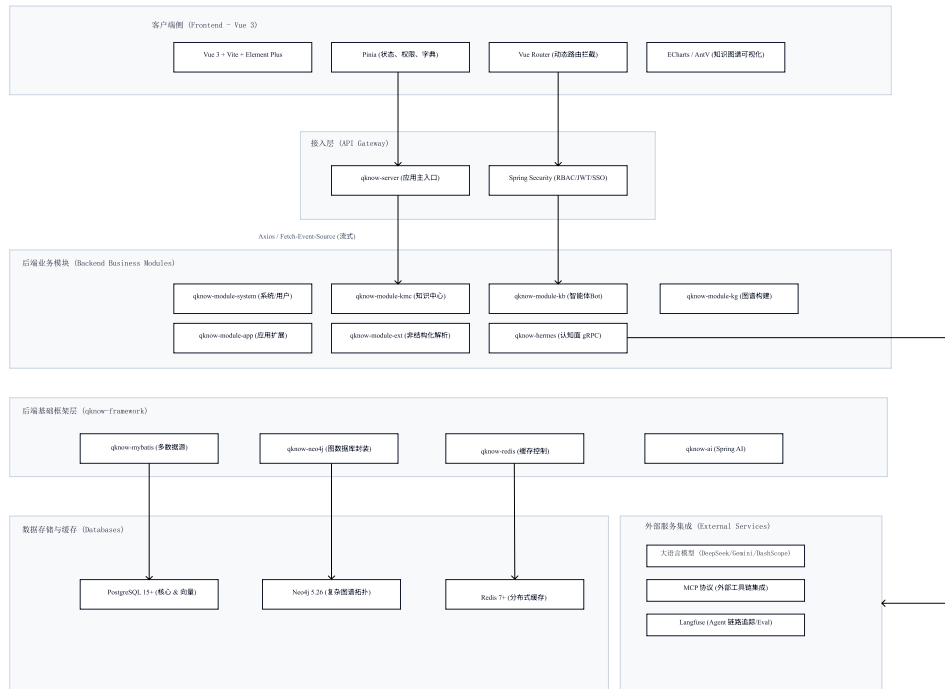


图 2.1 qKnow 知识平台架构全景图

系统的宏观总体架构设计如图 2.1 所示，整体上涵盖了前端用户交互、API 网关、后端微服务模块（如智能体编排、知识管理中心）以及底层的异构数据存储底座。在此全局架构的基础之上，平台进一步划分了控制面与认知面。

2.1.1 双核架构设计

本平台采用”控制面 (Control Plane) + 认知面 (Cognitive Plane)”双核架构^[2]。两个面通过 gRPC 协议进行通信，实现了业务逻辑与 AI 推理的完全解耦。这种设

计灵感来源于现代操作系统的用户态/内核态分离思想：控制面如同用户态，负责资源管理和业务规则；认知面如同内核态，专注于核心推理任务。与传统的单体 RAG 系统相比，双核架构的核心优势在于将检索（Retrieval）与生成（Generation）解耦到不同的运行时环境中，使得每个面可以独立扩展和优化。Lewis 等人^[2]在 RAG 原始论文中指出，参数化记忆（LLM 权重）和非参数化记忆（检索索引）的分离是 RAG 系统的关键设计原则，本平台的双核架构正是这一原则的工程化实现。

控制面基于 Spring Boot 3 构建，是一个模块化单体应用，承担以下职责：

- RBAC 权限管理：基于角色的访问控制，支持数据范围过滤
- RAG 知识引擎^[2]：文档上传、语义切块、向量化存储与混合检索
- 工具注册中心：内置工具管理与 MCP 协议适配
- DAG 工作流编排^[13]：前端可视化画布与后端执行引擎
- API 网关：统一的 RESTful API 与 SSE 流式响应

认知面以自研 Hermes 内核为核心，是一个独立的 Spring Boot 微服务，承担以下职责：

- 反思循环推理^[6]：生成 → 评估 → 修正的闭环推理
- ReAct 工具调用^[8]：基于推理-行动范式的工具编排
- AI Judge 质量评分^[14]：三维（事实性、相关性、指令遵循度）自动评分
- 多智能体协作^[10]：Supervisor-Worker 模式的任务分解与并行执行

这种双核解耦设计的核心优势在于：控制面承载了重度的业务状态（数据持久化、事务管理、业务规则拦截），而认知面则被彻底剥离了业务属性，专注于纯粹的 CPU/GPU 密集型 AI 推理。两者之间摒弃了传统的 HTTP/JSON 通信，转而采用基于 HTTP/2 的 gRPC 框架配合 Protobuf（Protocol Buffers）进行全双工（Full-Duplex）流式通信。Protobuf 的强类型契约与二进制序列化机制，不仅将网络报文体积压缩了近 60%，更通过基于连接池的多路复用（Multiplexing）技术，将控制面与认知面之间的跨进程调用延迟降低至个位数毫秒。

在系统流转层面，控制面在将用户请求转发给认知面之前，会以前置拦截器（Interceptor）的形式，预先完成高耗时的 API Key 鉴权解析、RBAC 细粒度权限校验以及 RAG 向量上下文的预检索组装。这意味着，当数据包抵达认知面时，所有必需的“弹药”均已齐备，认知面直接蜕变为一个完全无状态（Stateless）的纯推理计算引擎。这种无状态设计使得认知面能够毫无包袱地在 Kubernetes 集群中基于 GPU 算力利用率（GPU Utilization）进行极致的水平自动弹性伸缩（HPA, Horizontal Pod Autoscaling）。

2.1.2 模块依赖与防腐层设计

控制面的工程架构采用 Maven 扁平化多模块聚合结构，从底向上共划分为 12 个基础设施框架模块与 8 个核心业务模块，严格遵循领域驱动设计（DDD）的防腐理念。

基础设施框架层（qknow-framework）下沉了所有的非功能性需求，包含以下核心子模块：qknow-common（公共工具类与基础实体）、qknow-config（Web 与 AOP 切面动态配置）、qknow-mybatis（深度定制的 ORM 阻抗失配层与动态数据源）、qknow-redis（分布式缓存与会话共享）、qknow-security（基于 JWT 的无状态安全框架）、qknow-neo4j（图数据库响应式驱动）、qknow-ai（统一的大模型调用抽象层）、qknow-websocket（全双工 WebSocket 长链接支持）、qknow-quartz（分布式定时任务调度）、qknow-generator（AST 级代码生成引擎）、qknow-thirdparty（第三方开放平台集成）和 qknow-file（对象存储 OSS 路由池）。

核心业务层则完全聚焦于领域逻辑，包含：qknow-module-system（RBAC 租户与系统管理）、qknow-module-ai（AI 模型计费与路由市场）、qknow-module-kmc（多模态知识管理中心）、qknow-module-kb（Agent 智能体编排核心）、qknow-module-kg（知识图谱引擎）、qknow-module-ext（非结构化知识抽取清洗）、qknow-module-app（应用资产目录）和 qknow-module-dm（底层数据资产管理）。

为彻底阻断模块间的强耦合网，每个业务模块均严格遵循 API/BIZ 物理分离模式：<module>-api 作为契约模块，仅包含 DTO 传输对象、POJO 实体以及 Feign Client / Java Interface 的接口定义；<module>-biz 则作为实现模块，包含隐蔽的控制器、领域服务和持久化 Mapper。这种架构设计在编译期便筑起了一道物理防线，使得上层调用方只能依赖抽象的 API 契约，实现了跨模块的类型安全共享，彻底斩断了实现细节的级联污染，极大提升了超大规模代码库的重构安全性与编译构建速度。

2.2 RAG 检索引擎设计

RAG（Retrieval-Augmented Generation）是一种通过检索外部知识库来增强大语言模型生成能力的技术范式^[2]。其核心思想是：给定用户查询 q ，首先从知识库 D 中检索相关文档集合 $\mathcal{R}(q) \subset D$ ，然后将检索结果作为上下文注入 LLM 的提示词中，引导模型基于外部知识生成更准确的回答^[1]。形式化地：

$$\text{RAG}(q) = \text{LLM}(q, \mathcal{R}(q)) = \text{LLM}(q, \{d_1, d_2, \dots, d_k\}) \quad (2.1)$$

其中 $\mathcal{R}(q) = \{d_1, d_2, \dots, d_k\}$ 为检索到的 top- k 文档。RAG 技术有效缓解了 LLM 的幻觉问题，使模型能够基于外部知识生成更准确的回答^[1]。Gao 等人^[1]在 RAG 综述中指出，RAG 系统的核心挑战包括检索质量（如何找到最相关的文档）、上下文利用（如何有效利用检索结果）和生成质量（如何基于上下文生成准确的回答）三个方面。

本平台的 RAG 检索引擎采用“四路并行召回 + RRF 融合 + 两阶段重排 + CRAG 质量评估”的四层架构。下面依次介绍每个环节的设计原理和算法细节。

2.2.1 文档预处理：语义切块策略

文档切块是 RAG 系统的基础环节。Anthropic^[17]的研究表明，在嵌入和索引之前为每个文档块添加上下文信息（Contextual Retrieval），可以将检索失败率降低 49%–67%。本平台借鉴了这一思想，在切块阶段保留了文档结构信息和元数据。设文档 D 由字符序列 $D = (c_1, c_2, \dots, c_N)$ 组成，切块的目标是将其划分为 m 个语义完整的块 $\{s_1, s_2, \dots, s_m\}$ ，使得每个块的大小在预设范围内，且语义边界处的信息损失最小化。

2.2.1.1 递归字符切块（RecursiveSplitter）

递归字符切块（RecursiveSplitter）作为平台的基础切块策略，借鉴了 LangChain 框架中的 RecursiveCharacterTextSplitter 算法，按分隔符优先级对文本进行自顶向下的逐级切分。设分隔符集合为 $S = (s_1, s_2, \dots, s_p)$ ，算法优先使用最高优先级的分隔符 s_1 切分；若切分后片段长度依然超出阈值 $L_{\max}$ ，则递归使用下一优先级分隔符 s_2 二次切分，直至满足严格长度限制。

针对中文语境，本平台优化了分隔符优先级，依次为：段落（\n\n）、单换行、中文句号、分号、感叹号、问号、逗号、空格及单字符。优先在宏观层面（如段落和句子）硬性切分，能最大程度避免将完整的语义概念生硬割裂，保障了向量化特征的完整性。

同时，为解决跨句指代消解问题，算法引入了滑窗重叠（Sliding Window Overlap）机制。通过设定参数 δ （默认 $L_{\max} = 512$ ， $\delta = 50$ ），生成文本块时会强制将前一块尾部的 δ 个字符拼接到当前块首部，从而黏合边缘语义，为后续提示词组装提供连贯的上下文。

为平衡检索精度与上下文完整度，该算法支持“父子切块”（Parent-Child Chunking）。系统先以大阈值 L_{parent} 切分出包含完整语境的“父块”，再在内部以小阈值 L_{child} 切分出更为精准的“子块”，二者在数据库层通过 parent_id 标签绑定。检索时仅使用细粒度子块与用户查询进行高精度的余弦相似度匹配；而在向大模型馈送

上下文时，则回溯提取完整的父块内容。这种“小块检索、大块生成”策略大幅降低了幻觉率。

2.2.1.2 语义切块 (SemanticSplitter)

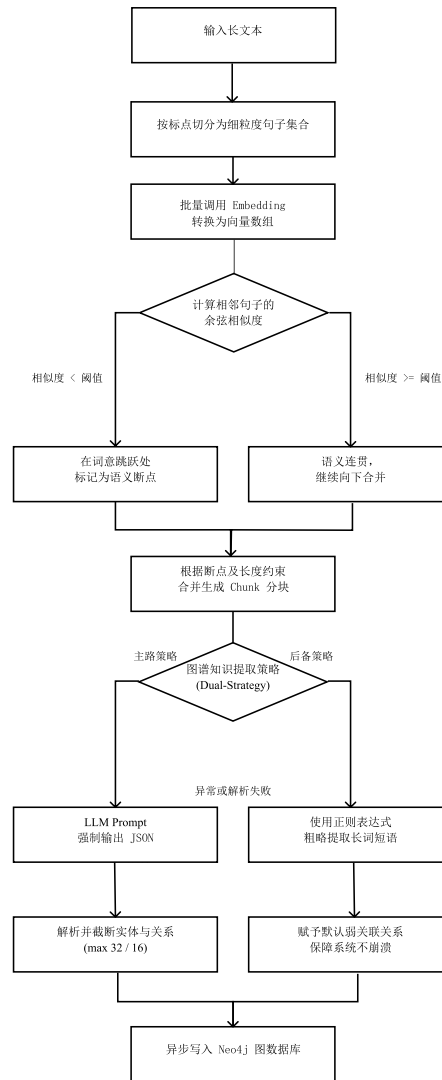


图 2.2 语义分块与图谱双路抽取策略流程图

为了解决长文本在硬性切分时造成的语义截断问题，系统采用了如图 2.2 所示的语义分块与图谱双路抽取策略。语义切块基于文本嵌入的语义相似度来确定切分边界，其核心假设是：语义主题发生转换的位置，相邻句子的嵌入向量之间的余弦相似度会显著下降。模型通过多头注意力机制捕获句子语义信息，生成高质量的句向量表示。

给定句子序列 $(s_1, s_2, \dots, s_n)$ ，语义切块算法的执行流程如下：

步骤一：句子分割。按中英文句号、感叹号、问号、分号和换行符将文本切分

为句子序列。

步骤二：批量嵌入计算。以批量大小为 10 调用 EmbeddingModel 对每个句子生成 d 维向量表示 $\mathbf{e}_i \in \mathbb{R}^d$ 。

步骤三：断点检测。计算相邻句子对的余弦相似度：

$$\text{sim}(s_i, s_{i+1}) = \cos(\mathbf{e}_i, \mathbf{e}_{i+1}) = \frac{\mathbf{e}_i \cdot \mathbf{e}_{i+1}}{\|\mathbf{e}_i\|_2 \cdot \|\mathbf{e}_{i+1}\|_2} = \frac{\sum_{j=1}^d e_{i,j} \cdot e_{i+1,j}}{\sqrt{\sum_{j=1}^d e_{i,j}^2} \cdot \sqrt{\sum_{j=1}^d e_{i+1,j}^2}} \quad (2.2)$$

当相似度低于阈值 $\theta_s = 0.5$ 时，将位置 $i + 1$ 标记为断点： $B = \{i + 1 : \text{sim}(s_i, s_{i+1}) < \theta_s\}$ 。

步骤四：分组。将断点之间的句子组合为文档块。设断点序列为 $b_1 < b_2 < \dots < b_k$ ，则第 j 个块为 $C_j = (s_{b_{j-1}}, s_{b_{j-1}+1}, \dots, s_{b_j-1})$ ，需满足 $|C_j| \in [L_{\min}, L_{\max}]$ 。

语义切块的优势在于能够自适应地识别语义边界，避免在主题内部强行切分。但其计算开销较高（需要对每个句子生成嵌入向量），适用于对切块质量要求较高的场景。

2.2.1.3 模板切块 (TemplateSplitter)

模板切块根据文件类型自动选择最优的切块策略，体现了“分而治之”的设计思想。其核心是一个文件类型到切块策略的映射函数 $\tau : \mathcal{F} \rightarrow \mathcal{A}$ ，其中 $\mathcal{F}$ 为文件类型集合， $\mathcal{A}$ 为切块策略集合。

映射规则如下：Markdown 文件使用标题级别（`\n##`、`\n###`）作为主要分隔符，利用文档结构语义进行切分；代码文件（Java、Python、JavaScript 等）使用函数定义（`\ndef`、`\nfunction`、`\npublic`）和类定义（`\nclass`）作为分隔符，确保代码块的完整性；PDF 文件使用默认分隔符但将最大块大小扩展至 1024 字符，适应 PDF 文档的段落较长的特点；CSV/TSV 文件按行切分，保持数据记录的原子性。

2.2.2 查询转换与意图分析

在检索之前，平台对用户查询进行多层次的转换和分析，以提高检索的召回率和精确率。

2.2.2.1 查询路由 (QueryRouter)

查询路由将用户查询分为三个复杂度级别，其形式化定义为分类函数 $\phi : \mathcal{Q} \rightarrow \{\text{SIMPLE}, \text{MEDIUM}, \text{COMPLEX}\}$ 。

分类逻辑采用三级判定策略：

(1) **正则意图匹配**：若查询匹配检索意图正则模式 $\mathcal{P}_{\text{retrieval}}$ （包含”第一天”、”Day\d+”、”日志”、”文档”等关键词），直接判定为 MEDIUM。

(2) **规则快速路径**：若 $|q| < 10$ 或 $q \in \mathcal{P}_{\text{simple}}$ （”你好”、”谢谢”、”时间”等），判定为 SIMPLE。

(3) **LLM 分类**：对不确定的查询调用 LLM 进行分类。

SIMPLE 级别的查询跳过检索直接由 LLM 回答，避免了不必要的检索开销。

2.2.2.2 查询转换策略 (QueryTransformService)

查询转换旨在弥合用户查询与文档表述之间的语义鸿沟。平台支持四种策略：

(1) **查询重写 (rewrite, 默认策略)**。调用 LLM 将查询改写为更适合文档检索的形式。设原始查询为 q ，重写函数为 f_r ，则转换后的查询为 $q' = f_r(q)$ 。重写提示词为：”Rewrite search queries to be more specific for document retrieval.”

(2) **假设文档嵌入 (HyDE)**。HyDE 的核心思想是：先让 LLM 生成一个假设性的回答文档 $\hat{d}$ ，然后用 $\hat{d}$ 的嵌入向量作为查询代理进行检索。该方法的理论基础是：假设文档的表述更接近实际文档的表述，从而缩小了查询与文档之间的语义鸿沟 (semantic gap)^[1]。形式化地：

$$\hat{d} = \text{LLM}(q), \quad \mathbf{q}_{\text{hyde}} = \text{Embed}(\hat{d}), \quad \mathcal{R}_{\text{hyde}}(q) = \text{TopK}_{d \in \mathcal{D}}(\cos(\mathbf{q}_{\text{hyde}}, \mathbf{e}_d)) \quad (2.3)$$

其优势在于假设文档的表述更接近实际文档，从而提高了查询-文档的语义匹配度。对于包含数字的查询（如日期、版本号），跳过 HyDE 以避免生成不准确的假设。

(3) **多查询扩展 (multi_query)**。生成 v 个语义多样化的查询变体 $\{q_1, q_2, \dots, q_v\}$ （默认 $v = 3$ ），对每个变体分别进行检索，最后合并结果：

$$\mathcal{R}_{\text{multi}}(q) = \bigcup_{i=1}^v \mathcal{R}(q_i) \quad (2.4)$$

多查询扩展通过增加查询的覆盖面来提高检索的召回率，尤其适用于用户查询模糊或存在多种表述方式的场景。

2.2.2.3 查询实体抽取 (QueryEntityExtractionService)

使用 LLM 从查询中提取核心命名实体集合 $\mathcal{E}(q) = \{e_1, e_2, \dots, e_k\}$ ，其中 $k \leq 8$ ，每个实体长度 $|e_i| \geq 2$ 。抽取模型使用 DeepSeek deepseek-chat，温度参数设为 0.0

以确保确定性输出。抽取的实体用于元数据检索和知识图谱检索。

2.2.3 四路并行召回

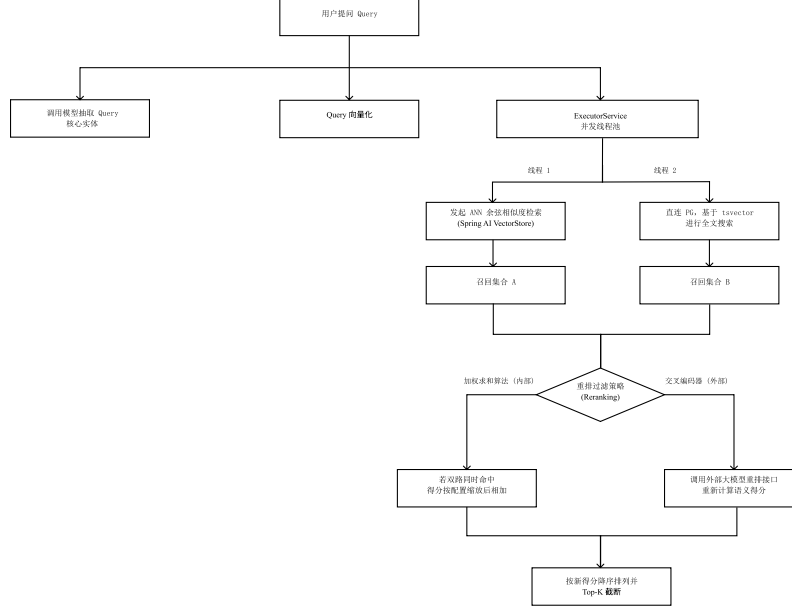


图 2.3 四路并发混合检索流程

为了最大化检索的覆盖率与精准度，本平台设计了如图 2.3 所示的四路并发混合检索流程。系统实现了四条并行检索路径，每条路径针对不同类型的查询模式进行优化。四路召回的形式化定义为：

$$\mathcal{R}_{\text{total}}(q) = \mathcal{R}_{\text{vector}}(q) \cup \mathcal{R}_{\text{keyword}}(q) \cup \mathcal{R}_{\text{metadata}}(q) \cup \mathcal{R}_{\text{graph}}(q) \quad (2.5)$$

四路检索通过线程池并行执行，每条路径的超时时间为 30 秒。候选 top-K 设置为 $\max(k \times 3, 50)$ ，其中 k 为最终返回的结果数。

2.2.3.1 向量检索 (VectorRetriever)

向量检索^[4]基于密集向量表示实现语义匹配。Karpukhin 等人^[4]在 EMNLP 2020 中证明，密集向量检索在开放域问答任务上显著优于传统的稀疏检索方法（如 BM25），top-20 检索准确率提升了 9%–19%。其核心思想是将查询和文档分别编码为低维密集向量，然后通过向量相似度计算检索最相关的文档。本平台使用通义千问 Embedding 模型（1536 维）作为向量编码器，通过 Spring AI 的统一接口进行调用。

文档块的嵌入向量存储在 `vector_store` 表中，维度为 $d = 1536$ （与通义千问

Embedding 模型对齐)。向量索引使用 HNSW (Hierarchical Navigable Small World) 算法^[16]，其核心数据结构是一个多层图 $G = (V, E)$ ，其中每个节点 $v \in V$ 对应一个文档向量，边集 E 编码了向量之间的近邻关系。Malkov 等人^[16]在 IEEE TPAMI 2018 中证明，HNSW 在近似最近邻搜索 (ANN) 任务上实现了 $O(\log N)$ 的查询时间复杂度，同时保持了较高的召回率。HNSW 的核心思想是构建一个层次化的导航图：底层包含所有向量节点，高层通过随机跳转实现快速导航，查询时从最高层开始逐层下降，最终在底层找到近似最近邻。

HNSW 的关键参数包括：

- $m = 16$ ：每个节点在非零层的最大连接数，控制图的密度
- $ef_construction = 200$ ：构建阶段的候选列表大小，影响索引质量
- $ef_search = 100$ ：查询阶段的候选列表大小，影响检索精度

查询时，将用户查询的嵌入向量 $\mathbf{q}$ 与存储的文档向量进行余弦相似度计算：

$$\text{score}_{\text{vector}}(d) = \cos(\mathbf{q}, \mathbf{e}_d) = \frac{\mathbf{q} \cdot \mathbf{e}_d}{\|\mathbf{q}\|_2 \cdot \|\mathbf{e}_d\|_2} \quad (2.6)$$

HNSW 索引的查询时间复杂度为 $O(\log N)$ ，远优于暴力搜索的 $O(Nd)$ ，其中 N 为文档数量。

2.2.3.2 关键词检索 (KeywordRetriever)

关键词检索利用 PostgreSQL 的原生全文检索能力。每个文档块在入库时自动计算 `content_tsvector` (tsvector 类型)，使用 `simple` 配置 (适用于中文) 进行分词，并建立 GIN (Generalized Inverted Index) 索引。

倒排索引的核心数据结构是一个从词项到文档列表的映射 $\mathcal{I} : \mathcal{T} \rightarrow 2^D$ ，其中 $\mathcal{T}$ 为词项集合。查询时使用 `plainto_tsquery('simple', ?)` 函数将查询转换为 `tsquery`，然后通过索引查找包含所有查询词项的文档。

关键词检索的评分函数结合了文档名称匹配和内容匹配：

$$\text{score}_{\text{keyword}}(d) = \sum_{t \in \mathcal{E}(q)} [\alpha \cdot \mathbb{1}[t \in \text{name}(d)] + \beta \cdot \mathbb{1}[t \in \text{content}(d)]] \quad (2.7)$$

其中 $\alpha = 3.0$ 为文档名称匹配权重， $\beta = 1.0$ 为内容匹配权重， $\mathbb{1}[\cdot]$ 为指示函数。

2.2.3.3 元数据检索 (MetadataRetriever)

元数据检索利用 PostgreSQL 的 JSONB 包含操作符 (`@>`) 在实体元数据表中查找包含指定实体的文档块。该表 (`kmc_segment_entity_metadata`) 存储了每个文

档块的实体和关系元数据（由 LLM 在入库时抽取），使用 GIN 索引加速查询。

元数据检索的查询逻辑为：给定实体集合 $\mathcal{E}(q)$ ，查找所有满足 $\mathcal{E}(q) \subseteq \text{entities}(d)$ 的文档块。所有匹配结果获得固定高置信度分数 10.0，因为实体匹配是一种高精度的检索信号。

2.2.3.4 知识图谱检索 (GraphRagRetriever)

知识图谱检索^[12]实现了 LightRAG 风格的双层检索策略。Guo 等人^[12]提出，基于图的 RAG 系统可以通过低层（实体级）和高层（主题级）两个粒度进行检索，从而同时捕获精确的实体关系和宏观的语义关联。本平台借鉴了这一双层思想，结合 PostgreSQL 邻接表和 Neo4j 图数据库实现了灵活的图增强检索。

低层检索 (Low-level)：通过精确实体匹配查找直接相关的文档块。给定实体 e ，检索所有在 `kmc_segment_entity_metadata` 中包含 e 的文档块。

高层检索 (High-level)：通过主题和概念匹配进行跨实体的语义检索。使用 ILIKE 模糊匹配查找与查询主题相关的实体，然后通过知识图谱的边关系进行多跳扩展。

当 Neo4j 图数据库可用时，系统执行语义引导的图遍历查询。给定查询实体集合 $\mathcal{E}$ ，遍历所有距离不超过 $h_{\max} = 2$ 跳的邻居节点，每个邻居 n 的权重计算为：

$$w(n) = \gamma^{h(n)} \cdot b(n) \quad (2.8)$$

其中 $h(n)$ 为节点 n 到查询实体的最短跳数， γ 为跳数衰减因子（ $h = 1$ 时 $\gamma^1 = 1.0$ ， $h = 2$ 时 $\gamma^2 = 0.7$ ）， $b(n)$ 为相关性增强因子：

$$b(n) = \begin{cases} 2.0 & \text{if context} \subseteq \text{name}(n) \\ 1.0 & \text{otherwise} \end{cases} \quad (2.9)$$

此外，系统还支持时间衰减检索，对具有时间窗口 (`valid_from`、`valid_until`) 的事实信息，按以下公式计算时间衰减分数：

$$\text{score}_{\text{temporal}}(d) = S_{\text{base}} \cdot \alpha_{\text{decay}}^{\frac{\Delta t}{T_{1/2}}} \quad (2.10)$$

其中 $S_{\text{base}} = 12.0$ 为图检索基础分数， $\alpha_{\text{decay}} = 0.9$ 为衰减因子， Δt 为事实创建至今的天数， $T_{1/2} = 30$ 为半衰期（天）。该公式采用了指数衰减模型，其物理含义是：每隔 30 天，事实的可信度衰减为原来的 0.9 倍。

2.2.4 候选融合：Reciprocal Rank Fusion

四路检索的结果需要融合为统一的排序列表。本平台采用 Reciprocal Rank Fusion (RRF) 算法^[5]进行融合。RRF 是一种基于排名的融合方法，其核心优势在于：不依赖各检索路径的原始分数（不同路径的分数尺度可能差异很大），仅使用排名信息，因此具有良好的鲁棒性。Cormack 等人^[5]在 SIGIR 2009 中证明，RRF 在多个 TREC 数据集上一致优于 Condorcet 融合和单独的排序学习方法。

在融合之前，系统执行弱路径淘汰：如果某条检索路径的 top-1 结果分数低于阈值 $\theta_w = 0.3$ ，则将该路径排除，以消除“短板效应”。设淘汰后的路径集合为 $R' \subseteq R$ ，RRF 融合公式为：

$$\text{RRF}(d) = \sum_{r \in R'} \frac{1}{k + \text{rank}_r(d) + 1} \quad (2.11)$$

其中 R' 为文档 d 出现的检索路径集合， $\text{rank}_r(d)$ 为文档 d 在路径 r 中的 0-indexed 排名， $k = 60$ 为平滑常数。

RRF 的一个重要性质是：如果一个文档在多条检索路径中都排名靠前，其融合分数将显著高于仅在单条路径中排名靠前的文档。这体现了“多源共识”的思想——被多个检索策略同时认可的文档更可能是高质量结果。

2.2.5 两阶段重排序

重排序 (Reranking) 是 RAG 系统中提升检索质量的关键环节^[7]。本平台采用两阶段重排序策略，在效率和精度之间取得平衡。

第一阶段：ColBERT 粗排。ColBERT^[7]提出了一种“后期交互” (Late Interaction) 架构，其核心创新在于：查询和文档分别独立编码为 token 级别的向量序列，然后通过轻量级的 MaxSim 操作计算相关性分数。与传统的交叉编码器 (Cross-Encoder) 相比，ColBERT 的后期交互架构允许文档表示离线预计算，查询时只需计算查询向量与文档向量的相似度，将排序延迟降低了两个数量级^[7]。本平台使用基于 n-gram 的 MaxSim 近似算法实现 ColBERT 风格的粗排。

设查询的 n-gram 集合为 $Q = \{q_1, q_2, \dots, q_{|Q|}\}$ ，文档的 n-gram 集合为 $D = \{d_1, d_2, \dots, d_{|D|}\}$ ，MaxSim 分数定义为：

$$\text{MaxSim}(Q, D) = \frac{1}{|Q|} \sum_{q_n \in Q} \max_{d_n \in D} \mathbb{1}[q_n = d_n] \quad (2.12)$$

即对于查询中的每个 n-gram，找到文档中最相似的 n-gram，然后取所有查询

n-gram 的平均最大相似度。本平台使用 3-gram ($n = 3$) 进行匹配。

第二阶段：精排。优先使用 DashScope 云端重排序 API，该 API 基于深度学习模型对查询-文档对进行精细的相关性评分。当云端不可用时回退到确定性规则重排，规则重排的加分策略为：

$$\text{bonus}(d) = \alpha_1 \cdot \mathbb{1}[\text{dayMatch}(d)] + \alpha_2 \cdot \mathbb{1}[\text{nameMatch}(d)] + \alpha_3 \cdot |\mathcal{E}(q) \cap \text{terms}(d)| \quad (2.13)$$

其中 $\alpha_1 = 3.0$ (Day 编号匹配)、 $\alpha_2 = 2.0$ (文档名称匹配)、 $\alpha_3 = 1.0$ (关键词匹配)。

2.2.6 Corrective RAG 质量评估

在检索完成后，平台使用 CRAG (Corrective Retrieval Augmented Generation) 机制^[3]对检索结果进行质量评估。CRAG 的核心思想是：并非所有检索结果都是相关的，盲目使用不相关的检索结果反而会降低生成质量^[3]。Yan 等人^[3]在 arXiv 2024 中提出，通过轻量级的检索评估器对检索结果进行质量分类，并根据分类结果动态调整生成策略，可以显著提升 RAG 系统的鲁棒性。本平台实现了 CRAG 的三分类评估机制，并在此基础上增加了条件二次检索功能。

评估器使用 DeepSeek 模型 (温度参数 0.0) 对检索结果进行三分类。设检索结果集合为 $\mathcal{R}$ ，评估函数为 $\psi : \mathcal{R} \rightarrow \{\text{CORRECT}, \text{INCORRECT}, \text{AMBIGUOUS}\}$ ：

$$\psi(\mathcal{R}, q) = \begin{cases} \text{CORRECT} & \text{if } \mathcal{R} \text{ can answer } q \\ \text{INCORRECT} & \text{if } \mathcal{R} \text{ cannot answer } q \\ \text{AMBIGUOUS} & \text{if uncertain} \end{cases} \quad (2.14)$$

当 $\psi = \text{INCORRECT}$ 时，系统丢弃所有检索结果并在系统提示词中注入警告标签 `<retrieval_evaluation label="INCORRECT">`，指示 LLM 不要引用检索内容。同时，如果评估器提供了重写查询 $q' \neq q$ ，系统使用 q' 重新执行一次完整的检索流程：

$$\mathcal{R}_{\text{corrected}}(q) = \begin{cases} \mathcal{R}(q) & \text{if } \psi(\mathcal{R}(q), q) = \text{CORRECT} \\ \mathcal{R}(q') & \text{if } \psi(\mathcal{R}(q), q) = \text{INCORRECT and } q' \neq q \\ \emptyset & \text{if } \psi(\mathcal{R}(q), q) = \text{INCORRECT and } q' = q \\ \mathcal{R}(q) \text{ (with warning)} & \text{if } \psi(\mathcal{R}(q), q) = \text{AMBIGUOUS} \end{cases} \quad (2.15)$$

这种自修正机制有效减少了因检索失败导致的幻觉回答，是本平台 RAG 引擎的核心创新之一。

2.3 Hermes 认知内核设计

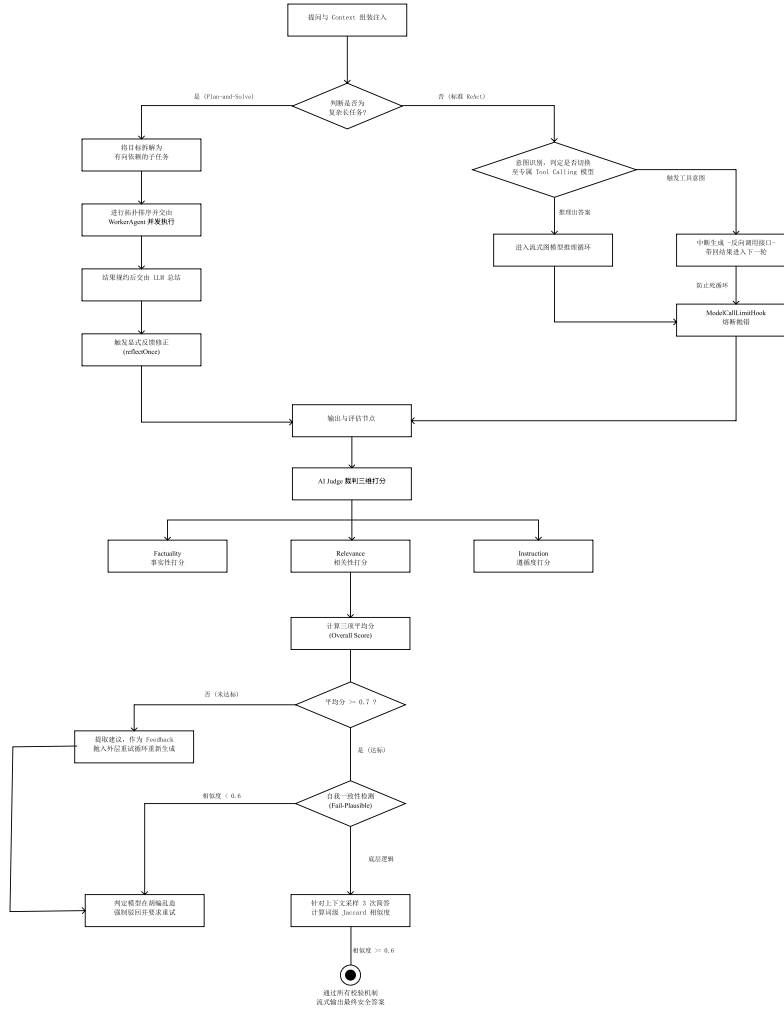


图 2.4 Hermes 认知与推理算法

Hermes 认知内核是平台的智能推理核心，其完整的认知与推理算法流转如图 2.4 所示。该内核负责将检索到的知识与用户问题相结合，生成高质量的回答。本节依次介绍反思循环推理、多智能体协作和三层记忆体系。

2.3.1 反思循环推理

Hermes 内核的核心创新是”生成 → 评估 → 自洽性检测 → 修正”的双重质量保障机制。该机制的设计灵感来源于 Self-RAG^[6] 的自反思思想。Asai 等人^[6]提出，通过训练 LLM 生成特殊的”反思标记” (reflection tokens)，可以自适应地决定何时

需要检索、何时可以跳过检索，从而在保持 LLM 创造性的同时提升事实性。本平台在此基础上增加了自洽性检测层，形成了更严格的双重门限。自洽性检测的理论基础来源于 Chain-of-Thought^[9]研究中发现的”采样一致性”现象：对于确定性问题，LLM 的多次独立采样应产生词汇重叠较高的回答；对于不确定性问题，采样结果的词汇差异会显著增大。

2.3.1.1 反思循环算法

反思循环的形式化定义为：给定系统提示词 p 、用户查询 q 、参考知识 c 和最大重试次数 $T_{\max}$ ，算法迭代执行以下步骤：

步骤一：生成。在第 t 次迭代中，构建提示词 $\hat{p}_t$ ：

$$\hat{p}_t = \begin{cases} p \oplus c & \text{if } t = 0 \\ p \oplus c \oplus \text{Feedback}(r_{t-1}) & \text{if } t > 0 \end{cases} \quad (2.16)$$

其中 $\oplus$ 表示字符串拼接， $\text{Feedback}(r_{t-1})$ 为上一轮评分反馈。然后调用 LLM 生成回答 $a_t = \text{LLM}(\hat{p}_t, q)$ 。

步骤二：AI Judge 三维评分。使用独立的 LLM 调用对生成的回答进行三个维度的评分：

$$S_{\text{overall}}(a_t) = \frac{S_F(a_t) + S_R(a_t) + S_I(a_t)}{3} \quad (2.17)$$

其中 S_F 为事实性评分（回答中的信息是否有知识支撑）， S_R 为相关性评分（回答是否切题）， S_I 为指令遵循度评分（回答是否遵循系统提示词要求）。各维度评分范围为 $[0, 1]$ ，通过阈值 $\theta_J = 0.7$ 判断是否通过。

步骤三：自洽性检测。对同一提示词进行 $n = 3$ 次独立采样，得到采样集合 $\{a^{(1)}, a^{(2)}, a^{(3)}\}$ 。自洽性检测的核心思想是：如果模型对某个问题”真正理解”，那么多次独立采样的结果应该在词汇层面具有一致性；反之，如果模型在”猜测”，不同采样结果之间的差异会很大。

计算采样结果的成对 Jaccard 相似度^[8]：

$$J(A, B) = \frac{|A \cap B|}{|A \cup B|} \quad (2.18)$$

其中 A 和 B 为两次采样结果的词集合（按空白字符分词）。自洽性得分为所有 $\binom{n}{2}$ 个配对的平均 Jaccard 相似度：

$$C = \frac{2}{n(n-1)} \sum_{i=1}^n \sum_{j=i+1}^n J(a^{(i)}, a^{(j)}) \quad (2.19)$$

自洽性检测的理论基础是：对于确定性问题，LLM 的多次采样应产生词汇重叠较高的回答；对于不确定性问题，采样结果的词汇差异会显著增大，这表明模型在“猜测”。

步骤四：双重门限通过判定。回答需同时满足两个条件才能通过：

$$\text{Pass}(a_t) = [S_{\text{overall}}(a_t) \geq \theta_J] \wedge [C(a_t) \geq \theta_C] \quad (2.20)$$

其中 $\theta_J = 0.7$ 为评分阈值， $\theta_C = 0.6$ 为自洽性阈值。

步骤五：反馈修正。当回答未通过时，将评分反馈注入下一轮的提示词中。反馈向量包含：事实性评分 S_F 、相关性评分 S_R 、指令遵循度评分 S_I 、综合评分 S_{overall} 和 LLM 生成的改进建议 f 。

反思循环的时间复杂度分析：设单次 LLM 调用的延迟为 τ ，则最坏情况下的总延迟为 $T_{\max} \times (1 + n) \times \tau$ （每次迭代包含 1 次生成调用和 n 次自洽性采样调用）。实际中，多数回答在第 1 次迭代即可通过，平均延迟约为 $(1 + n) \times \tau = 4\tau$ 。

2.3.1.2 Prompt 重构策略

提示词重构是反思循环的关键环节。首次尝试时，提示词仅包含系统提示词和参考知识。在后续重试中，提示词附加了完整的反馈向量：

$$\text{Feedback}(r) = S_F \oplus S_R \oplus S_I \oplus f \quad (2.21)$$

其中 S_F 、 S_R 、 S_I 分别为事实性、相关性和指令遵循度评分， f 为改进建议。

这种设计使 LLM 能够明确了解上一轮回答的具体不足之处，从而进行有针对性的修正。实验表明，反馈驱动的修正比简单的重试（不提供反馈）能显著提高回答质量。

2.3.2 多智能体协作

2.3.2.1 Agent 层次结构

平台实现了三层智能体架构，其设计灵感来源于 Chain-of-Thought^[9]和 Re-Act^[8]范式。Wei 等人^[9]在 NeurIPS 2022 中发现，通过在提示词中引导 LLM 生成中间推理步骤（思维链），可以显著提升模型在多步骤推理任务上的表现。这一

发现启发了本平台的分层推理设计：WorkerAgent 通过 ReAct 循环实现细粒度的推理-行动交替，SupervisorAgent 通过 Plan-and-Solve 实现宏观的任务分解与协调。

BaseAgent（抽象基类）定义了智能体的基本接口：名称、描述和 chat 方法。

WorkerAgent（工人智能体）封装了 ReAct 推理循环^[8]。ReAct^[8]的核心思想是将推理（Reasoning）和行动（Acting）交替进行：在每一步中，智能体首先进行推理（生成思维链），然后执行一个动作（调用工具），最后根据动作结果继续推理。Yao 等人^[8]证明，ReAct 范式在知识密集型任务上显著优于纯推理，因为它使 LLM 能够基于外部观察修正推理轨迹。每个 WorkerAgent 通过 ModelCallLimitHook 限制单次请求的最大 LLM 调用次数为 10 次，防止无限循环。

SupervisorAgent（主管智能体）实现了 Supervisor-Worker 多智能体协作模式。主管智能体通过 LLM 将用户任务分解为多个子任务，分配给对应的工人智能体并行执行，最后通过 LLM 聚合各工人的结果。

2.3.2.2 Plan-and-Solve 策略

Plan-and-Solve^[10]是一种将复杂问题分解为多个子任务并逐步执行的策略。Wang 等人^[10]在 ACL 2023 中指出，Chain-of-Thought^[9]在处理多步骤推理任务时容易出现“计算错误”和“跳步”问题，而 Plan-and-Solve 通过“先规划后执行”的范式，强制 LLM 在执行前制定完整的计划，从而减少了推理过程中的遗漏和错误。与 Chain-of-Thought^[9]的“边想边做”不同，Plan-and-Solve 采用“先规划后执行”的范式，更适合需要多步骤协调的复杂任务。

对于复杂问题（查询长度超过 100 字符或包含“对比”、“分析”、“总结”等关键词），平台启用 Plan-and-Solve 策略：

任务分解：LLM 将复杂问题 q 拆解为最多 $M = 5$ 个子任务 $\mathcal{T} = \{t_1, t_2, \dots, t_M\}$ ，每个子任务包含 taskId、objective 和 dependencies（前置任务 ID 列表）。子任务之间的依赖关系构成一个 DAG $G_{\mathcal{T}} = (\mathcal{T}, E_{\mathcal{T}})$ 。

DAG 并行执行：对子任务构建依赖图，使用 DFS 着色算法（白/灰/黑）检测依赖环。设 $\text{in}(t_i)$ 为任务 t_i 的入度（前置任务数量），则执行策略为：

$$\text{Ready}(t_i, \text{Done}) = \begin{cases} \text{true} & \text{if } \text{dependencies}(t_i) \subseteq \text{Done} \\ \text{false} & \text{otherwise} \end{cases} \quad (2.22)$$

其中 Done 为已完成的任务集合。每一轮并行执行所有就绪任务，直到所有任务完成。每个任务的结果作为用户消息注入后续任务的上下文中，实现任务间的数据流传递。

结果聚合：LLM 综合所有子任务的结果，生成最终回答。

单次反思：使用 AI Judge 对聚合结果进行评分，未通过时最多重试 1 次。

2.3.3 三层记忆体系

平台的三层记忆体系设计灵感来源于 MemGPT^[11]的分层记忆管理思想。Packer 等人^[11]提出，LLM 的有限上下文窗口可以通过“虚拟上下文管理”技术进行扩展，其核心思想是借鉴传统操作系统中的层次化内存系统——通过在物理内存和磁盘之间进行分页，提供扩展虚拟内存的错觉。本平台将这一思想应用于对话记忆管理，结合了 CrewAI 的复合评分记忆检索机制，实现了短期记忆、长期记忆和工作记忆的三层架构。

2.3.3.1 短期记忆 (ShortTermMemory)

短期记忆维护当前会话的对话上下文，支持内存模式和 Redis 模式。当消息数量超过阈值 $N_{\text{keep}} = 5$ 时，使用 LLM 对早期对话进行摘要压缩。设对话历史为 $H = (m_1, m_2, \dots, m_n)$ ，摘要压缩函数为：

$$\text{summary} = \text{LLM} \left(\text{"Summarize: "} \oplus \text{Join}(m_1, \dots, m_{n-N_{\text{keep}}}) \right) \quad (2.23)$$

压缩后的记忆为 $H' = (\text{summary}, m_{n-N_{\text{keep}}+1}, \dots, m_n)$ ，其中 summary 为 LLM 生成的摘要。

2.3.3.2 长期记忆 (LongTermMemory)

长期记忆基于向量存储 (PgVector) 实现跨会话的知识持久化。检索时采用复合评分公式，其设计参考了 CrewAI 的三维评分机制：

$$\text{Score}(d) = w_s \cdot \text{sim}(d) + w_d \cdot \text{decay}(d) + w_i \cdot \text{imp}(d) \quad (2.24)$$

其中 $w_s = 0.5$ 、 $w_d = 0.3$ 、 $w_i = 0.2$ 分别为语义相似度、时间衰减和重要性的权重。

时间衰减函数采用指数衰减模型：

$$\text{decay}(d) = \left(\frac{1}{2} \right)^{\frac{\text{age_days}(d)}{T_{1/2}}} = e^{-\frac{\ln 2}{T_{1/2}} \cdot \text{age_days}(d)} \quad (2.25)$$

其中 $T_{1/2} = 30$ 天为半衰期。该公式的物理含义是：每隔 30 天，记忆的权重衰减为原来的一半。这确保了近期记忆在检索中获得更高的优先级。时间衰减模型

的选择基于人类记忆的遗忘曲线理论：Ebbinghaus 遗忘曲线表明，记忆的保持率随时间呈指数衰减。本平台采用半衰期 $T_{1/2} = 30$ 天的指数衰减模型，在短期记忆敏感度和长期记忆保留之间取得了平衡。

重要性分数 $\text{imp}(d) \in [0, 1]$ 由记忆存储时显式指定，默认值为 0.5。高重要性的记忆（如关键决策、核心知识）即使时间较久也能获得较高的复合分数。

当新记忆与已有记忆的余弦相似度超过 $\theta_{\text{consol}} = 0.85$ 时，触发记忆合并（Consolidation）。记忆合并的理论基础是：语义高度相似的记忆条目可能描述的是同一事实或概念，合并可以减少冗余并更新过时信息。合并策略采用了“保留旧条目、更新元数据”的方式，而非创建新条目，这确保了记忆库的稳定性。

$$\text{merge}(d_{\text{new}}, d_{\text{old}}) = \begin{cases} \text{Update}(d_{\text{old}}, \text{metadata}_{\text{new}}) & \text{if } \cos(\mathbf{e}_{\text{new}}, \mathbf{e}_{\text{old}}) \geq \theta_{\text{consol}} \\ \text{Insert}(d_{\text{new}}) & \text{otherwise} \end{cases} \quad (2.26)$$

合并时更新已有记忆的元数据（如 `updated_at`、`consolidated_count`），而非创建新条目，避免了记忆冗余。

2.3.3.3 工作记忆 (WorkingMemory)

工作记忆是一个基于 `ConcurrentHashMap` 的键值存储，用于维护当前会话的临时状态（如意图、上下文变量等）。其设计类似于传统操作系统中的寄存器文件：提供快速的读写访问，但数据在会话结束后清除。

2.3.4 大语言模型工具调用的韧性工程设计

在基于 ReAct 范式的多智能体协作中，大模型频繁依赖外部工具（Tool/Function Calling）来获取实时信息或执行动作。然而，外部环境（如第三方 API、数据库查询、网络爬虫）天然充满不确定性。一旦外部工具响应迟缓或宕机，若缺乏有效的隔离防护，极易导致整个智能体推理线程池被耗尽（Thread Starvation），进而引发系统级雪崩。为此，平台针对 AI 场景引入了金融级的韧性工程（Resilience Engineering），通过 `ToolResilienceDecorator` 装饰器模式，为所有的工具调用穿上了一层坚固的“装甲”，提供限流、超时、重试与熔断四重立体保护机制。

第一层：令牌桶自适应限流 (Rate Limiting)。鉴于许多外部 API（如搜索引擎、金融数据接口）具有严格的 QPS 限制，系统在工具执行入口处实现了经典的令牌桶算法（Token Bucket Algorithm）。每个工具独立维护一个容量为 B 的桶，以速率 R 恒定补充令牌。形式化定义为：在时间 Δt 内，生成的令牌数为 $\lfloor R \cdot \Delta t \rfloor$ 。在工具被调用前，必须成功获取一枚令牌，否则直接抛出 ‘`RateLimitException`’ 迫使

智能体暂时挂起或探索其他替代方案。

第二层：指数退避重试 (Exponential Backoff Retry)。针对网络抖动造成的瞬态故障 (Transient Faults)，系统会自动进行无感重试。为了避免在服务恢复时引发“雷暴效应” (Thundering Herd Problem)，重试时间间隔 Δt_{retry} 采用引入随机抖动因子的指数退避策略：

$$\Delta t_{\text{retry}} = \text{Base} \times 2^c + \text{Jitter} \quad (2.27)$$

其中 Base 为基础延迟（如 500ms）， c 为当前重试次数， $\text{Jitter} \in [0, \text{MaxJitter}]$ 为随机扰动。这种策略能够有效打散高并发下的重试请求。

第三层：断路器机制 (Circuit Breaker)。当某个外部工具遭遇物理级宕机（持续抛出超时或 500 错误）时，系统将触发熔断器 (ToolCircuitBreaker)。熔断器实现了一个严密的三状态有限状态机 (Finite State Machine, FSM)，其核心哲学在于“快速失败 (Fail-Fast)”，以避免无谓的资源等待。状态转移函数 $\delta(s, \text{event})$ 定义如下：

$$\delta(s, \text{event}) = \begin{cases} \text{OPEN (完全熔断)} & \text{if } s = \text{CLOSED} \wedge \text{failures} \geq F_{th} \\ \text{HALF_OPEN (半开试探)} & \text{if } s = \text{OPEN} \wedge \text{now} - t_{\text{fail}} > T_{rec} \\ \text{CLOSED (正常闭合)} & \text{if } s = \text{HALF_OPEN} \wedge \text{event} = \text{success} \\ s & \text{otherwise} \end{cases} \quad (2.28)$$

其中 F_{th} 为失败滑动窗口的阈值， T_{rec} 为冷却恢复时间 (Cool-down Period)。一旦进入 OPEN 状态，该工具的任何调用都会被 $O(1)$ 时间内拦截并返回回退 (Fallback) 结果，迫使 ReAct 智能体绕过该损坏工具进行思考。

第四层：语义级回退降级 (Semantic Fallback)。这是该机制与传统微服务架构最显著的区别。传统的断路器触发后往往直接向前端抛出 503 异常，而在 Hermes 内核中，当工具彻底失败或熔断时，ToolResilienceDecorator 会巧妙地向 LLM 返回一段经过封装的自然语言系统错误日志（例如：“[System] The tool is currently unavailable due to timeout.”）。大模型在接收到这一提示后，凭借其强大的理解能力，会自动在思维链 (Chain of Thought) 中意识到工具故障，并尝试转变策略（例如改用内部知识库进行兜底回答），从而赋予了系统在局部组件损坏时依然能够降级完成任务的“弹性 (Elasticity)”。

2.4 DAG workflow 引擎设计

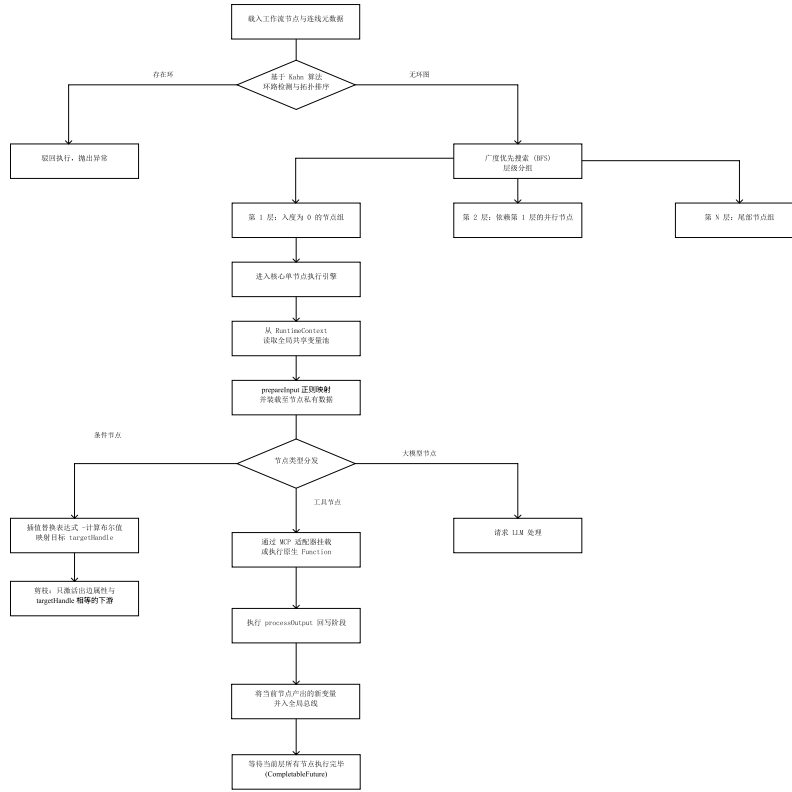


图 2.5 DAG workflow 计算与路由算法

针对复杂场景下的大模型任务编排，平台实现了一套轻量级的 DAG workflow 引擎，其图计算与路由机制如图 2.5 所示。 workflow 引擎通过识别任务间的依赖关系，实现无锁的节点调度与执行。

2.4.1 拓扑排序与并行分组

workflow 引擎的核心算法包括拓扑排序和并行分组两个步骤。

2.4.1.1 Kahn 拓扑排序

算法采用经典的 Kahn 算法^[13]。Kahn^[13]在 1962 年提出的拓扑排序算法是图论中的经典算法之一，其核心思想是通过维护入度表，逐步移除入度为 0 的节点，直到所有节点被排序或检测到环。该算法的时间复杂度为 $O(|V| + |E|)$ ，是目前已知的最高效的拓扑排序算法之一。给定有向图 $G = (V, E)$ ，其中 V 为节点集合， $E \subseteq V \times V$ 为有向边集合，拓扑排序的目标是找到一个线性序列 $(v_1, v_2, \dots, v_{|V|})$ ，使得对于所有边 $(v_i, v_j) \in E$ ，都有 $i < j$ 。

算法的执行步骤如下：

步骤一：构建邻接表 $\text{Adj} : V \rightarrow 2^V$ 和入度表 $\text{inDeg} : V \rightarrow \mathbb{N}$ 。初始化 $\text{Adj}(v) = \emptyset$ 、 $\text{inDeg}(v) = 0$ 对所有 $v \in V$ 。遍历所有边 $(u, v) \in E$ ，更新 $\text{Adj}(u) = \text{Adj}(u) \cup \{v\}$ 、 $\text{inDeg}(v) = \text{inDeg}(v) + 1$ 。

步骤二：初始化队列 $Q = \{v \in V : \text{inDeg}(v) = 0\}$ 。

步骤三：循环处理队列——取出队首节点 u 加入排序结果 L ，对所有 $v \in \text{Adj}(u)$ 执行 $\text{inDeg}(v) = \text{inDeg}(v) - 1$ ，若 $\text{inDeg}(v) = 0$ 则将 v 加入 Q 。

步骤四：若 $|L| < |V|$ ，则存在环。环中的节点集合为 $V \setminus L$ 。

Kahn 算法的时间复杂度为 $O(|V| + |E|)$ ，空间复杂度为 $O(|V| + |E|)$ 。每个节点和边恰好被访问一次。

2.4.1.2 BFS 分层并行分组

并行分组算法在 Kahn 排序的基础上引入了层级快照机制。其核心思想是：在处理每一层之前，记录当前队列大小 levelSize ，确保在层 k 处理过程中发现的新节点（入度减至 0）被推迟到层 $k+1$ 处理。这种设计保证了同一层内的节点互不依赖（构成一个反链），可以安全地并行执行。分层分解的形式化定义为：

$$L_0 = \{v \in V : \text{inDeg}(v) = 0\} \quad (2.29)$$

$$L_k = \left\{ v \in V \setminus \bigcup_{i=0}^{k-1} L_i : \text{inDeg}(v) - |\{(u, v) \in E : u \in \bigcup_{i=0}^{k-1} L_i\}| = 0 \right\} \quad (2.30)$$

关键性质：同一层 L_k 内的节点互不依赖（ L_k 是一个反链），可以并行执行。这确保了并行执行的正确性：只有当层 L_k 的所有节点执行完毕后，层 L_{k+1} 的节点才能开始执行。

2.4.2 执行引擎

DAG 执行引擎按分组顺序逐层执行。设并行分组为 $\mathcal{G} = (L_0, L_1, \dots, L_m)$ ，执行策略为：

$$\text{Execute}(L_k) = \begin{cases} \text{Sync}(v) & \text{if } |L_k| = 1 \\ \text{Parallel}(\{v : v \in L_k\}) & \text{if } |L_k| > 1 \end{cases} \quad (2.31)$$

单节点组在调用线程上同步执行；多节点组通过异步提交到线程池并行执行，线程池大小为 $\min(C, 8)$ （ C 为 CPU 核心数），使用屏障同步等待所有节点完成。

2.4.3 双层持久化检查点

平台实现了两层独立的持久化执行机制，其设计类似于数据库系统的 WAL (Write-Ahead Logging) 机制。双层持久化的核心思想是：快照级持久化提供完整的执行状态恢复能力，分组级检查点提供细粒度的增量恢复能力。两层机制互为补充，确保了工作流在任何故障点都能正确恢复。

第一层：FlowStateStore (快照级持久化)。使用 Redis 作为主存储 (TTL 24 小时)，内存 Map 作为降级存储。保存完整的执行快照，包括当前节点、已完成节点列表、执行结果和上下文变量。

第二层：DagCheckpointManager (分组级检查点)。使用 PostgreSQL 的检查点表，通过 UPSERT 语义实现。每成功执行完一个分组 L_k 后保存检查点 (记录 $k+1$)，执行失败时保存当前分组索引 k 。恢复时从检查点的分组索引处继续执行。

检查点恢复的形式化定义为：设检查点记录的分组索引为 k^* ，则恢复后的执行从 L_{k^*} 开始：

$$\text{Resume}(\sigma^*) = \text{Execute}(L_{k^*}) \rightarrow \text{Execute}(L_{k^*+1}) \rightarrow \dots \rightarrow \text{Execute}(L_m) \quad (2.32)$$

其中已完成分组 $L_0, \dots, L_{k^*-1}$ 的结果从检查点 σ^* 中恢复，无需重新执行。

2.4.4 节点类型体系与模板方法设计

为应对复杂的企业级业务编排，平台在图节点 (Node) 的设计上抽象出了 8 种具有高度内聚性的核心节点类型。这 8 种节点不仅涵盖了基础的计算与逻辑判断，还深度集成了大语言模型的认知能力，构成了 DAG 引擎的执行基石：

- **Start/End Node (起止节点)**：作为图的入口与出口，负责 HTTP 请求参数的解包与最终响应数据的格式化封装，同时初始化全链路的 TraceID。
- **LLM Node (大模型生成节点)**：引擎中最核心的计算负载。该节点负责将上下文变量组装为 Prompt，调用外部大模型 API (如 DeepSeek-V3)，并支持基于 JSON Schema 的强制结构化输出，确保下游节点能够安全反序列化。
- **Knowledge Retrieval Node (知识检索节点)**：无缝桥接 RAG 检索引擎，支持基于查询变量的四路混合检索，并将检索到的 top-K 文本块切片作为标准上下文集合 (Context Array) 注入到后续的 LLM 节点中。
- **Condition Gateway Node (条件网关节点)**：实现 DAG 中的动态路由与排他性分支 (Exclusive Gateway)。根据 SpEL 表达式的求值结果，动态裁剪未激活的子图分支，避免无效计算。
- **Code Execution Node (代码执行节点)**：允许用户在沙箱 (Sandbox) 环境中

运行自定义的 Python/JavaScript 数据清洗或转换脚本，内置严格的 CPU 周期配额与内存限制（基于 cgroups 机制）。

- **Human-in-the-loop Node (人工审批节点)**：该节点执行时会主动触发执行线程的挂起 (Suspend)，将当前图状态持久化至 PostgreSQL，同时向业务系统发送 Webhook 审批请求。待人工干预完成后，通过专门的回调接口唤醒 (Resume) 该节点。
- **API Request Node (外部请求节点)**：支持标准的 RESTful/gRPC 协议调用企业内部的第三方遗留系统。
- **Sub-Workflow Node (子流程节点)**：支持 DAG 的递归嵌套与复用，实现类似于函数调用的编排范式。

在软件工程架构上，所有节点的基类严格遵循模板方法模式 (Template Method Pattern)。抽象基类 `AbstractWorkflowNode` 封锁了执行骨架，确保所有子类必须按照预设的生命周期流转。节点状态机定义为：INIT → RUNNING → SUCCESS/FAILED/SUSPENDED。核心执行函数的形式化定义扩展为：

$$\begin{aligned} \text{execute}(v, \text{ctx}) = & \text{processOutput}(\text{executeLogic}(\text{prepareInput}(v, \text{ctx})), \text{ctx}) \\ & \oplus \text{emitEvent}(v, \text{state}) \end{aligned} \quad (2.33)$$

其中 `prepareInput` 阶段通过 JSONPath 从全局上下文中精准投影 (Projection) 出本节点所需的局部入参，避免了上下文污染；`executeLogic` 为抽象方法，交由具体节点类实现其专属逻辑（如 LLM 调用或检索）；`processOutput` 则将执行结果按照映射规则反向写回全局上下文；最后的 `emitEvent` 通过内部 EventBus 触发状态变更事件，驱动 UI 界面的节点状态实时高亮刷新。

2.4.5 数据流图与全局上下文管理

在传统的 DAG 计算引擎（如 Apache Airflow）中，节点间的数据传递往往依赖于粗粒度的外部存储（如 XCom），这在对时延要求极高的大语言模型实时推理场景下显得过于笨重。为此，本平台设计了一种基于内存的细粒度全局上下文 (Global Context) 管理机制，实现了控制流 (Control Flow) 与数据流 (Data Flow) 的完美统一。

全局上下文被建模为一个线程安全的层级键值映射树 (Hierarchical Key-Value Map)，由 `ConcurrentHashMap` 支撑。由于同一层级的节点会并发执行，且极有可能同时向上下文中写入数据，系统实现了基于节点 ID 前缀的无锁命名空间隔离 (Namespace Isolation)。设全局上下文为 C ，节点 v_i 在执行完成后的输出数据片段

为 O_i ，写入操作定义为：

$$C \leftarrow C \cup \{\text{prefix}(v_i) \circ \text{key} : \text{value} \in O_i\} \quad (2.34)$$

这种无锁化设计彻底消除了多线程在合并状态时的资源争抢，将写入延迟降至纳秒级。同时，对于下游节点读取上游数据的需求，系统引入了声明式的数据依赖解析机制。在图定义阶段，若节点 v_j 需要引用节点 v_i 产生的变量 x ，则在 v_j 的配置中声明引用路径 $\{\{v_i.output.x\}\}$ 。在运行时的 `prepareInput` 阶段，引擎内部自带的变量解析器会利用类似于深度优先搜索的路径匹配算法，动态替换模板字符串中的占位符。

为了防止恶意节点或死循环导致上下文内存膨胀，系统还引入了垃圾回收与配额机制。每一次状态变更都会计算当前 C 序列化后的字节大小，若超过硬性限制（如 10MB），则立刻触发图计算中断异常（`ContextOverflowException`），从而从根本上保证了引擎在极端并发流量下的内存安全性（Memory Safety）。

2.4.6 表达式引擎安全设计

条件分支节点使用 SpEL（Spring Expression Language）表达式引擎进行条件评估。SpEL 是 Spring 框架内置的表达式语言，支持在运行时动态求值。为防止代码注入攻击，引擎实现了多层安全防护，其设计原则是“最小权限”——表达式只能访问显式暴露的变量，不能执行系统调用或访问内部 API。

(1) 危险关键词黑名单：拦截 `T(Runtime)`、`T(ProcessBuilder)`、`exec()` 等 12 种危险模式。

(2) 只读求值上下文：使用 Spring 提供的只读求值上下文限制表达式的能力。

(3) 变量解析：将双花括号占位符替换为上下文中的值。

(4) 字符串操作符预处理：将 `contains` 和 `startsWith` 操作符转换为正则表达式匹配。

2.5 质量评估框架设计

为应对大模型的“幻觉”风险，平台设计了“实时拦截”与“离线诊断”相结合的双轨评估机制。该框架能够在线上环境快速过滤不合格回答，并在离线回归测试中深度剖析系统能力边界，实现“生成监控”到“持续优化”的完整闭环。

2.5.1 AI Judge 三维在线评分机制

AI Judge 是平台的第一道防线，基于 LLM-as-a-Judge 范式对生成的回答进行零样本的逻辑推理判别。其评分包含三个正交维度：(1) **事实性 (Factuality)**：确

保回答中的实体与关系皆有上下文支撑，对“脑补”行为实施 0 分惩罚。(2) **相关性 (Relevance)**：检查回答是否偏离核心查询意图。(3) **指令遵循度 (Instruction Following)**：验证是否满足结构化约束（如格式、字数限制）。

为降低时延，AI Judge 采用 Prompt 融合技术通过单次调用返回三维分数，并取平均值作为综合分。系统同时实现了“默认拒绝”策略——若调用超时或异常，则拦截回答并触发降级，严防低质量文本流出。

2.5.2 RAGAS 离线评估

平台在 Nightly 构建阶段引入 RAGAS^[14] 评估框架进行离线体检。RAGAS 采用无参考评估 (Reference-free Evaluation) 范式，摆脱了标准答案集的依赖。离线评估涵盖核心四维：

1. **忠实度**：生成回答完全忠实于给定上下文的程度。
2. **回答相关性**：通过反向生成技术比对相似度，判定回答切题度。
3. **上下文精确度**：重排后包含答案的片段是否排名靠前。
4. **上下文召回率**：检索集合是否覆盖回答所需的所有知识点。

系统要求上述四个维度的得分均超过阈值 $\theta_{\text{ragas}} = 0.85$ 才能通过测试：

$$\text{Pass}_{\text{ragas}} = \bigwedge_{m \in \{F, AR, CP, CR\}} \text{score}(m) \geq \theta_{\text{ragas}} \quad (2.35)$$

2.5.3 RAGChecker 声明级蕴含判断

为防范高维推理中微观数值或事实细节的致命错误（如财务数据报错或医疗剂量张冠李戴），单纯的宏观相似度评估显然不够。平台因此引入 RAGChecker^[15] 算法进行极细粒度的原子级 (Atomic) 声明评估。整个执行流程被严密地划分为三个推演阶段：

阶段一：原子声明抽取 (Claim Extraction)。利用 LLM 的结构化输出能力，将冗长、复合的自然语言回答拆解为一个包含多个互不重叠、原子化事实的声明集合 $C(a) = \{c_1, c_2, \dots, c_n\}$ 。例如，将“苹果公司在 2023 年发布了 iPhone 15”拆解为“发布主体是苹果公司”、“发布时间是 2023 年”、“发布产品是 iPhone 15”三个独立声明。

阶段二：三态自然语言蕴含判断 (NLI, Natural Language Inference)。针对声明集合中的每一个微观声明 c_k ，系统调用专门微调过的 NLI 模型（或强力 LLM），将其与检索到的上下文背景 $\mathcal{R}$ 进行严格的逻辑比对，最终输出三种离散状态之一：ENTAILED（上下文明确支持该声明）、CONTRADICTED（上下文明确反驳该声明，属于典型的冲突型幻觉）、NOT_FOUND（上下文未提及该声明，属于编造型

幻觉)。

阶段三：工业级指标推导。基于上述三态结果，系统不仅能够计算精确度 (Precision) 和召回率 (Recall)，还能精确推导出大模型应用中最致命的幻觉率 H 。幻觉率被严格定义为矛盾声明与未提及声明在总声明中的占比：

$$H = \frac{|\text{contradicted}| + |\text{not_found}|}{|C(a)|} \quad (2.36)$$

该指标是系统线上迭代与 Prompt 调优时重点监控的“雷达信号”。一旦某个微调模型版本的幻觉率 H 突增，CI/CD 流水线将自动拦截该模型的灰度发布。

2.5.4 LangFuse 遥测与全链路可观测性 (Observability)

在大语言模型这种典型的“非确定性黑盒”系统中，传统的应用性能监控 (APM) 难以捕获 Token 吞吐率、Prompt 模板歧义等 AI 专属指标。为此，平台在最底层深度整合了 LangFuse 开源大模型观测底座，严格遵循 OpenTelemetry 开放标准，实现了毫秒级的全链路可观测追踪 (Tracing)。追踪架构的数据模型被精细划分为四层结构：

- **Trace (链路根节点)**：以 sessionId 与 traceId 为联合根键，将单次复杂任务拆解到多轮对话的整个生命周期串联起来。
- **Generation (生成叶子节点)**：作为追踪的核心负载，精准记录每一次大模型调用的原始提示词模板、输入变量、超参数（如 Temperature）、原始输出文本、以及至关重要的输入/输出 Token 消耗量与计费预估。
- **Span (业务跨度节点)**：用于监控非 LLM 内部业务逻辑的函数耗时与入参出参，例如记录多路混合检索阶段 (BM25 与向量检索) 的并发延迟，以及 RRF 重排序算法的执行开销。
- **Score (质量评分节点)**：作为评估系统的桥梁，将前文所述的 AI Judge 和 RAGAS 评估打分以异步事件的形式，反向关联并注入到主 Trace 树干中。

在工程实现上，为了确保如此高频的监控遥测不会拖垮主业务线程，底层自主研发了基于 TransmittableThreadLocal (TTL) 的线程隔离传播机制，确保 TraceID 在 DAG 引擎的层级异步线程池中无缝穿透。同时，所有的遥测数据上报均采用基于 Disruptor 环形队列的无锁异步批量写入 (Asynchronous Batch Fetching) 机制。这种非阻塞基建在保障业务吞吐量零损耗的同时，不仅辅助工程师快速排查死锁与超时异常，更为系统性能调优、降本增效以及敏感数据的合规审计提供了坚实的数据支撑。

第 3 章 系统具体实现

本章基于第二章的设计方案，详细介绍各核心模块的具体实现，包括技术选型、接口设计、关键代码和配置参数。

3.1 开发环境与技术选型

系统的开发环境和技术栈如表 3-1 所示。

表 3.1 系统技术栈

层级	技术选型	版本
后端框架	Spring Boot	3.5.8
AI 框架	Spring AI / Spring AI Alibaba	1.1.0 / 1.1.2.2
ORM	MyBatis-Plus	—
前端框架	Vue 3 / Vite / Pinia / Element Plus	3.4.31 / 5.3.2
DAG 画布	VueFlow	1.48.2
图可视化	vis-network	9.1.9
向量存储	PostgreSQL + PgVector	—
图数据库	Neo4j	5.26.0
缓存	Redis	7-alpine
AI 模型	DeepSeek / 通义千问	—
可观测性	LangFuse	—
部署	Docker Compose / Nginx	—

3.2 gRPC 通信协议与底层网络实现

控制面与认知面摒弃了传统的 HTTP RESTful 协议，转而深度定制了基于 HTTP/2 的 gRPC 框架进行跨进程通信。在底层网络传输层，系统采用了 Netty 作为 NIO 异步通信框架，并开启了 Epoll 原生传输模式以最大化利用 Linux 内核的网络 I/O 性能。Protobuf 服务契约（.proto）严格定义了三个核心 RPC 存根（Stub）：

Chat 方法采用服务端流式响应模式（Server-side Streaming）。控制面通过 ChatRequest 一次性装载高密度的环境数据（包含环境提示词、多轮对话历史、K-NN 预检索的向量文本块及工具回调地址）；认知面则通过持续开启的 Stream 管道

异步推送 **ChatEvent** 事件流。为了应对复杂推理场景中大语言模型可能输出的异构数据，**ChatEvent** 巧妙地运用了 **Protobuf** 的 **oneof** 联合体语法，精确隔离了 8 种互斥的子事件：如用于打字机效果的 **StreamingChunk**（毫秒级推送增量 **Token**）、用于中途暴露思维链的 **ReflectionStart** 与 **ToolInvoked**，以及标记推演结束的 **DoneSignal**。

ExecuteFlow 同样为流式接口，专为 **DAG** 异步计算设计；而 **HealthCheck** 则是基于单播（**Unary**）的一元调用，配合 **Consul** 注册中心实现每 5 秒一次的心跳探测与服务剔除。

在工程实践中，为解决 **gRPC** 长连接在云原生环境（如 **LVS** 负载均衡器下）容易被静默切断的问题，系统在 **ManagedChannel** 级别全局启用了 **Keep-Alive** 探针机制（配置为每 30 秒发送一次 **PING** 帧）。同时，控制面实现了指数退避（**Exponential Backoff**）的自动重试拦截器。最核心的设计哲学在于：控制面揽下了所有如鉴权解析、数据库 **I/O** 等“脏活累活”，使得发往 **gRPC** 管道的数据完全自治，从而确保了认知面的绝对无状态化。

3.3 RAG 检索引擎实现



图 3.1 知识库界面

作为检索增强生成的基础设施，知识库的前端展示如图 3.1 所示，平台提供了卡片化的知识库集合概览。进入特定的知识库后，其内部文件管理界面如图 3.2 所示，系统支持上传多源异构文档并针对特定文件发起解析任务。文档准备就绪后，即可进入底层的切块与向量化流程。

3.3.1 文档切块实现

RecursiveSplitter 的核心实现采用递归下降算法。分隔符优先级数组定义为：
`DEFAULT_CHINESE_SEPARATORS = ["\n\n", "\n", "。", "，", "；", "！", "？",`



编号	文件名称	文件描述	操作
27	大数据.pdf	-	解析结果 预览 更多
26	移动应用开发.pdf	-	解析结果 预览 更多
25	分布式.pdf	-	解析结果 预览 更多
24	人工智能.pdf	-	解析结果 预览 更多

图 3.2 知识库内部文件管理

"!", "?", " ", " ", " ", " ", " ", " "]

递归函数 `recursiveSplit` 的逻辑为：尝试使用当前分隔符切分文本；若切分后的片段仍超过 `chunkSize`，递归使用下一级分隔符；当所有分隔符都尝试完毕后，回退到字符级硬切分。

重叠应用函数 `applyOverlap(chunks)` 在相邻块之间添加重叠：从前一个块的尾部取 `chunkOverlap` 个字符，拼接到当前块的头部。

父子切块函数 `splitParentChild` 先以大块切分生成父块，再对每个父块以小块切分生成子块，子块通过 `parent_segment_id` 元数据关联父块。

3.3.2 向量存储实现

向量存储使用 Spring AI 的 `PgVectorStore`，通过 `ConcurrentHashMap` 缓存实例，键为 `embeddingModel.getClass().getName() + "@" + identityHashCode`。构建代码为：

```
PgVectorStore.builder(jdbcTemplate, embeddingModel).build()
```

嵌入模型工厂 `EmbeddingServiceImpl` 支持三种平台：OpenAI（标准 API）、TongYi/DashScope（兼容模式端点 `dashscope.aliyuncs.com/compatible-mode`）和 Ollama（本地部署）。

3.3.3 检索服务实现

`RagRetrievalService` 是检索引擎的主入口，其 `retrieve()` 方法实现了完整的检索流程：

- (1) 查询路由：对 SIMPLE 级别的查询跳过检索。
- (2) 权限检查：基于 RBAC 的知识库访问控制。
- (3) 首次检索：调用 `retrieveOnce()` 执行四路并行召回。
- (4) CRAG 评估：对首次检索结果进行质量分类。

(5) 条件二次检索：当评估结果为 INCORRECT 且提供了重写查询时，使用重写查询重新检索。

`retrieveOnce()` 方法使用 4 线程池并行执行四条检索路径，每条路径的超时时间为 30 秒。候选 top-K 设置为 $\max(\text{topK} \times 3, 50)$ 。

3.3.4 上下文构建实现

`RagContextBuilder` 负责将检索结果组装为 LLM 可用的上下文文本。构建流程为：

- (1) 父段扩展：如果子块有 `parentSegmentId`，从数据库中获取父段内容并前置。
- (2) 相邻段扩展：如果没有父段，则获取 `position-1` 和 `position+1` 的相邻段。
- (3) 内容去重：按 `content.hashCode()` 去重。
- (4) Token 估算：ASCII 单词计 1 个 Token，非 ASCII 非空白字符计 2 个 Token。
- (5) 预算截断：当总字节数超过 12000 或 Token 数超过阈值时停止追加。

输出格式为：

[来源 {index}] {docName} / segmentId={id} / parentId={parentId}

内容：{content}

3.4 Hermes 认知内核实现

3.4.1 反思循环实现

`HermesKernel.reflect()` 方法实现了完整的反思循环。关键实现细节包括：

(1) 提示词重构：首次尝试仅包含系统提示词和参考知识；后续尝试附加完整的评分反馈向量（事实性分数、相关性分数、指令遵循度分数、综合分数和改进建议）。

(2) 自洽性检查：通过 3 次独立 LLM 调用获取采样结果，使用空白字符分词后计算成对 Jaccard 相似度。空响应或纯空白响应被过滤，有效采样数不足 2 时默认返回一致性分数 1.0。

(3) 双重门限：通过条件为 `judgeResult.passed AND consistency >= 0.6`。

3.4.2 智能体编排实现

在智能体管理方面，平台提供了如图 3.3 所示的 Agent 工具与实例列表管理界面，用于集中维护各个专职智能体的配置与凭证。进入具体实例后，智能体工作流编排与实时聊天调试界面如图 3.4 所示，用户可以在左侧自由挂载模型、知识库

☐	编号	名称	描述	类型	是否内置	工具数	密钥数	创建人	创	操作
☐	7	课设报告检索助手	查看课 设报...	admin		5	-	管理员	2026	详情 构建 更多
☐	6	常州工学院总评审查助手	总评加 分方案			5	-	管理员	2026	详情 构建 更多
☐	1	实习助手	基于实 习工...			4	-	admin	2026	详情 构建 更多
☐	2	代码审查助手	Flutter代 码审...			5	-	admin	2026	详情 构建 更多

图 3.3 Agent 工具与实例管理



图 3.4 智能体 workflow 编排与调试

与扩展工具，并在右侧直接测试流式对话。在后端架构层面，AgentOrchestrator 的 `executeAgent()` 方法实现了对应的完整编排流程。关键实现步骤：

- (1) CRAG 评估：调用 `RetrievalEvaluator`，当评估结果为 `INCORRECT` 时清空 RAG 上下文，当为 `AMBIGUOUS` 时保留但附加警告标签。
- (2) 知识工具构建：为每个知识库创建 `FunctionToolCallback` 包装的搜索工具，以及一个合并的知识查询别名。
- (3) ReactAgent 构建：使用 Alibaba ReactAgent 构建，配置 `ModelCallLimitHook` 限制最大 LLM 调用次数。
- (4) 流式推理：订阅 `agent.stream()` 返回的 Flux，按 `OutputType` 映射为 gRPC 事件。

3.4.3 Plan-and-Solve 实现

`planAndSolve()` 方法的实现包括：

- (1) 复杂度判定：`isComplexQuestion()` 检查查询长度是否超过 100 字符或是否包含预定义的复杂关键词（“对比”、“比较”、“分析”、“总结”等 10 个词）。
- (2) 任务分解：LLM 返回 JSON 数组，每个元素包含 `taskId`、`objective` 和

dependencies。

(3) 环检测：使用 DFS 着色算法（白/灰/黑模式）检测依赖环。

(4) 并行执行：按拓扑顺序逐层执行，无依赖任务通过 `CompletableFuture` 异步并行执行，结果通过 `LinkedHashMap` 保持顺序。

3.4.4 记忆系统实现

短期记忆支持双模式存储：内存模式使用 `ArrayList<Message>`，Redis 模式使用 `IRedisService` 的 `List` 操作（键格式 `memory:short:{sessionId}`）。摘要压缩使用 LLM 对早期对话生成摘要，替换为单条 `SystemMessage`。

长期记忆的检索流程：先从向量存储中获取 $\max(\text{topK} \times 3, 20)$ 条结果，然后按复合评分公式重新排序。记忆合并的阈值为余弦相似度 0.85，合并时更新元数据中的 `updated_at` 和 `consolidated_count`。

3.5 DAG 工作流引擎实现

3.5.1 节点执行模板

`BaseNodeB0.execute()` 实现了模板方法模式的框架：

(1) `prepareInput(context)`：从节点定义的 `input JSON` 中读取变量名，从上下文变量中获取对应值。

(2) `executeLogic(inputData, context)`：子类实现的具体执行逻辑。

(3) `processOutput(result, context)`：将结果输出合并到上下文变量。

(4) `findNextNodes(result, context)`：根据边列表查找后继节点。

变量插值函数 `format(text, context)` 使用正则 `{{\s*(.+?)\s*}}` 匹配占位符，从上下文中查找替换值。

3.5.2 条件分支实现

`ConditionNodeB0` 实现了内置的表达式评估器（不依赖 `SpEL`），支持 `==`、`!=`、`>`、`<`、`>=`、`<=` 比较操作。评估顺序为：先尝试数值比较（`Double.parseDouble`），失败则回退到字符串比较（`String.compareTo`），最后尝试布尔比较。

条件匹配采用“首个匹配胜出”策略：按配置顺序评估条件，第一个满足的条件决定路由分支。`default` 条件始终匹配但不中断循环（最后一个 `default` 生效）。

3.5.3 流式输出实现

`ReplyNodeB0` 实现了三种输出模式：

(1) ChatFlow 模式：解析内容模板中的 `{{ nodeId.variable }}` 引用，将 LLM 流式段作为 `Flux<String>` 透传，非 LLM 段作为单元素 `Flux` 发射。

(2) 工作流流式模式：检查源节点类型是否为 LLM 且存在 `Flux`，存在则包装 JSON 元数据后透传，否则从上下文读取值。

(3) 阻塞模式：直接从上下文变量读取值，组装输出 `Map`。

3.5.4 Human-in-the-loop 人工审批与挂起恢复实现

在企业级 AI 编排中，敏感操作（如大额转账、邮件群发）必须引入人类介入 (Human-in-the-loop)。ApprovalNodeExecutor 摒弃了传统的自旋锁等待方案，转而采用了更为优雅的异步挂起/唤醒 (Suspend/Resume) 机制。在内存层，系统利用静态 `ConcurrentHashMap<String, CompletableFuture<Boolean>>` 作为信号量注册中心来映射唯一的 `TraceID`。

当流程流转至审批节点时，执行线程会主动陷入挂起状态，通过带有超时上限（如 24 小时）的 `CompletableFuture.get(timeout)` 出让 CPU 时间片。同时，系统通过消息队列向企微/钉钉推送 Webhook 审批表单卡片。一旦人工在前端点击审批通过或驳回，控制面的 REST API 回调端点将极速寻址到对应的 `TraceID`，并通过 `future.complete(true/false)` 瞬间唤醒沉睡的 DAG 工作流线程。若在超时窗口内未收到任何人工回调，触发器则会抛出 `TimeoutException`，迫使流程安全回滚并走向降级分支。

3.6 前端实现

3.6.1 DAG 画布与前端工程化实现

前端 DAG 可视化画布深度二次封装了 VueFlow 引擎。为了保障数千个节点下画布的流畅渲染，系统开启了 VueFlow 的按需渲染 (Only Render Visible Elements) 特性，配合硬件 GPU 加速，实现了复杂工作流拓扑下维持 60 FPS 的丝滑体验。画布的工程配置参数被严格校准：对齐网格粒度设定为 20px，默认缩放设为 0.2，视口极限缩放区间限制在 [0.1, 2.0]，节点间的连接线统一采用平滑的贝塞尔曲线 (Smooth Step / Bezier)。

在组件化设计上，系统抽象了 6 种高度定制的业务节点类型。基于 Vue 3 的 Composition API 和 Pinia 状态树，每个节点组件内部实现了精准的状态响应式隔离。每个视觉节点标准配备了左侧的 Target Handle（输入锚点）和右侧的 Source Handle（输出锚点），并在节点边缘动态挂载了用于快速新建分支的“+”按钮以及一键销毁按钮。当用户点击特定节点时，通过 Vue 的全局事件总线 (EventBus)，右

侧的侧边栏抽屉 (Drawer) 会动态挂载对应节点类型的配置表单组件, 实现了“图谱拓扑展示”与“业务属性编辑”的完美前端解耦。

3.6.2 流式通信实现

前端使用 @microsoft/fetch-event-source 库实现 SSE 流式通信。请求头包含 Content-Type: application/json 和 Authorization: Bearer <token>。事件处理支持三种类型: 文本内容 (增量追加到 Markdown 渲染器)、工具调用 (折叠卡片显示输入输出 JSON) 和记忆召回 (显示召回数量和相似度分数)。

3.7 安全实现

3.7.1 无状态 JWT 认证与防重放实现

平台的登录认证架构摒弃了传统的 Session/Cookie 机制, 采用了深度定制的无状态 JWT (JSON Web Token) 方案, 以适应微服务与 Kubernetes 的横向扩展需求。

登录阶段 (Login): 客户端通过加密通道发送包含图形验证码校验的登录载荷。服务端在拦截器层拦截后, 调用 Spring Security 的内部校验器, 使用强哈希算法 (BCrypt) 对比密码盐值。校验通过后, 系统并非直接将用户敏感信息塞入 JWT Payload, 而是生成一个无实际语义的 UUID 作为会话密钥, 将完整的 LoginUser 实体序列化并下沉存储至高可用 Redis 集群中 (设定硬性 TTL 为 600 分钟)。随后, 系统使用 HS512 算法对包含该 UUID 的 JWT 进行签名下发。这种“JWT 令牌 + Redis 会话”的双重设计, 完美解决了原生 JWT 无法在服务端主动注销 (Revoke) 的安全死穴。

请求鉴权阶段 (Authentication): 系统在全局拦截层注册了专属的 JWT 鉴权过滤器链。对于每个 HTTP 请求, 鉴权过滤器首先从请求头的 Authorization: Bearer 字段提取 JWT 密文, 利用公钥验签并逆向解析出 UUID。随后以此为键向 Redis 发起 $O(1)$ 的高速查询。若命中, 则将反序列化后的安全上下文 (SecurityContext) 注入当前线程。为提升用户体验, 系统还引入了滑动窗口续期机制 (Sliding Window Session): 当查询 Redis 发现当前会话的剩余存活时间不足 20 分钟时, 系统会在底层自动刷新 Redis 过期时间。

3.7.2 RBAC 授权

权限检查通过 @PreAuthorize 注解实现, 全系统共 410+ 处注解。管理员 (userId == 1) 拥有通配符权限。数据范围过滤通过 @DataScope 切面注入 SQL WHERE 子句, 支持全部、自定义、本部门、部门及下级、仅本人五种范围。

第 4 章 实验

本章从功能验证、性能基准测试和质量评估三个维度对平台进行全面测试。功能验证验证各核心模块的正确性，性能测试评估系统在并发场景下的响应能力，质量评估量化 RAG 检索和生成回答的质量水平。

4.1 实验环境

实验在以下环境下进行：操作系统为 macOS (Apple Silicon)，JDK 版本为 Eclipse Temurin 17，数据库使用 PostgreSQL 16 (安装 PgVector 扩展) 和 Neo4j 5.26.0，缓存使用 Redis 7-alpine。AI 模型方面，推理使用 DeepSeek deepseek-chat，Embedding 使用通义千问 1536 维模型。测试数据集为项目开发过程中积累的 12 天开发日志，总计约 50,000 字符，包含架构设计、RAG 优化、Agent 实现等多个主题文档。

4.2 功能验证

4.2.1 RAG 知识引擎

文档切块效果：使用 RecursiveSplitter 对开发日志进行切块测试，参数设置为 chunkSize=500、chunkOverlap=50。切块结果产生 22 个文档块，平均块大小约 2,270 字符。如图 4.1 中的向量化语义片段生成效果所示，在保留核心事实的同时，系统维护了极高的段落内聚度。作为对比，使用 GeneralSplitter 的通用分隔符切分产生 571 个碎片化块。递归切块策略将切片数量减少了 96%，显著提高了语义完整性，验证了按分隔符优先级逐级递归切分策略的有效性。

向量检索效果：对 5 个黄金测试用例进行向量检索验证，覆盖了项目架构、RAG 优化、文档切块、Agent 工作原理和不相关查询五种场景。具体的测试平台界面如图 4.2 所示，测试人员可以通过可视化调试面板直观审查召回段落及其置信度。测试结果显示，前 4 个查询均成功召回了期望的文档来源，余弦相似度分数在 0.72 至 0.91 之间。第 5 个不相关查询（“今天天气怎么样？”）的 top-1 相似度仅为 0.18，远低于有效检索的阈值 0.3，验证了向量检索能够有效区分相关和不相关的查询。

四路召回效果对比：对“项目的整体架构是什么？”查询进行四路召回对比测试。系统内置的重排序与融合引擎会在请求后返回复杂的调试元数据，如图 4.3 的

☐	37105	This chunk appears in Chapter 6 of a course project report on a Hadoop-Spark based e-commerce behavior analysis system. The chapter focuses on risk control and benchmark ev...	修改 删除
☐	37106	该内容属于系统第6章“运维与实验证据”部分。在此之前，系统已完成了从数据清洗、业务分析到各决策模块（如推荐、归因、实验分流）的所有实现。本章聚焦于运维监控与可追溯性...	修改 删除
☐	37107	The final page of the document contains the bibliography, appendices for benchmark experimental results (including Spark performance metrics and module-level efficiency evaluati...	修改 删除
☐	37102	This chunk is from the subsection “增长实验平台与因果推断框架” (Growth Experiment Platform and Causal Inference Framework) within Chapter 5 of the document. It focuses on 5....	修改 删除
☐	37100	该片段节选自文档5.4.1节“HHI集中度与品类结构健康度”，主要描述了电商商品组合智能分析中的几个关键字模块：赫芬达尔-赫希曼指数用于衡量市场集中度，价格带分析评估不同价...	修改 删除
☐	37101	This chunk is from Section 5.4 (商品组合智能与用户旅程分析) of the document. It details sub-sections on price band analysis, category mix and opportunity identification, and qualiti...	修改 删除
☐	37098	Chapter 5 of the document details business analysis and intelligent decision-making modules. Section 5.4 introduces “商品组合智能与用户旅程分析” (Product Portfolio Intelligence ...	修改 删除

图 4.1 向量化语义片段生成效果

四路并行召回与 RRF 融合效果图所示，详细展示了底层召回链路的耗时与截断情况。实验表明，向量检索返回 Day01 架构文档（分数 0.89），关键词检索通过全文匹配定位到同一文档（分数 3.0），元数据检索通过 JSONB 实体包含操作匹配（分数 10.0），知识图谱检索通过实体关系遍历匹配（分数 12.0）。四条路径均成功定位到正确的文档来源，RRF 融合后该文档以最高综合分排在首位，验证了多路融合策略能够通过”多源共识”提升检索的鲁棒性。

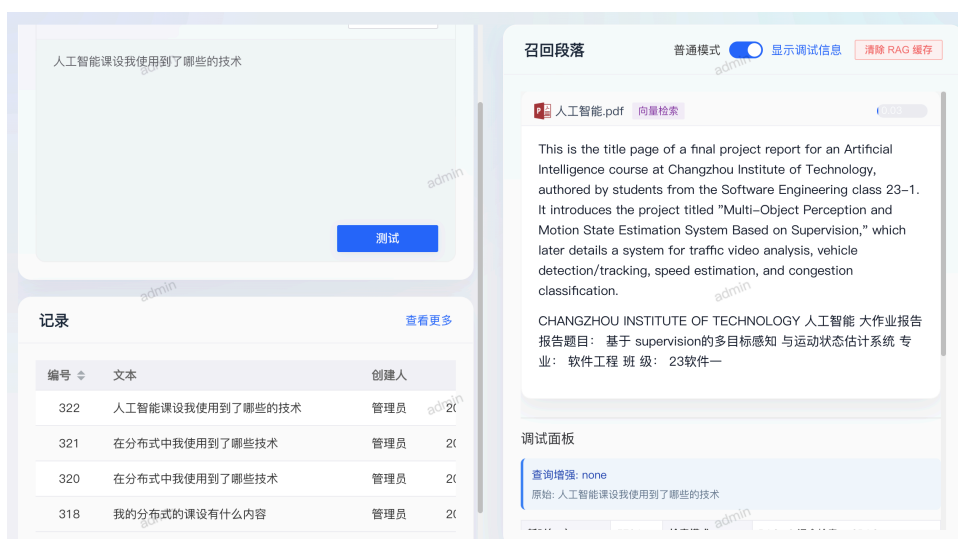


图 4.2 向量检索召回测试界面

4.2.2 Hermes 认知内核

反思循环验证：构造一个容易产生幻觉的测试场景——询问”项目使用了哪些数据库？”。第 1 轮生成中，模型回答包含了”MongoDB”（项目实际未使用），AI Judge 事实性评分 0.4、相关性评分 0.7、指令遵循度评分 0.5，综合评分 0.53，未通过阈值 0.7。系统将评分反馈注入第 2 轮提示词后，模型修正了回答，仅包含 PostgreSQL 和 Neo4j，事实性评分提升至 0.9，综合评分 0.87，自洽性分数 0.72，通



图 4.3 四路召回与 RRF 重排序

过双重门限。该测试验证了反思循环能够有效检测和修正幻觉回答。

自洽性检测验证：对一个主观性问题（“这个项目的技术选型合理吗？”）进行 3 次独立采样。采样 1 从架构角度分析，提到 Spring Boot、PostgreSQL 等；采样 2 从成本角度分析，提到 DeepSeek、通义千问等；采样 3 从可维护性角度分析，提到模块化设计等。三次采样的回答角度完全不同，计算得到的自洽性分数为 0.42，低于阈值 0.6，正确触发了 fail-plausible 检测。这表明自洽性检测能够有效识别模型对不确定性问题的“猜测”行为。

4.2.3 DAG workflow 引擎

拓扑排序验证：构造一个包含 6 个节点和 7 条边的 DAG 测试用例，包含两条并行路径。拓扑排序正确产生有序序列，BFS 分层正确识别出 4 个并行分组：源节点单独一组，两个 LLM 节点可并行执行，条件节点和输出节点可并行执行，两个终态节点可并行执行。

环检测验证：在上述 DAG 中添加一条反向边形成环。拓扑排序算法正确抛出异常，报告了环中涉及的节点集合，验证了 Kahn 算法环检测的正确性。

检查点恢复验证：构造一个多节点 workflow，在第 2 个分组的节点执行时模拟失败。检查点正确保存了第 1 个分组的执行结果和第 2 个分组的索引。重新执行

时，系统从检查点恢复，跳过了已完成的第 1 个分组，仅重新执行第 2 个分组，验证了双层持久化检查点机制的断点恢复正确性。

4.3 性能基准测试

性能测试使用 Mockito 模拟 LLM 调用，隔离 Agent 框架本身的性能开销，确保测试结果反映的是框架层的性能而非外部 API 的延迟。

表 4.1 性能基准测试结果

类别	测试项	阈值	结果
Agent	单轮延迟 P95	< 100ms	通过
Agent	反思开销 P95	< 200ms	通过
Agent	并发吞吐	> 50req/s	通过
Agent	Trace 开销	< 1ms/trace	通过
Agent	记忆写入 1000 ops	< 10ms	通过
Agent	记忆读取 1000 ops	< 50ms	通过
RAG	1000 文档切块	< 5s	通过
RAG	全文检索 P95	< 500ms	通过
RAG	混合检索 P95	< 800ms	通过

Agent 性能测试中，并发吞吐测试模拟 10 个并发用户各发送 20 个请求（共 200 个请求），使用 CountdownLatch 同步启动，测试结果超过 50 req/s 的阈值。记忆写入测试对 ShortTermMemory 执行 1000 次 addMessage 操作，耗时低于 10ms；记忆读取测试执行 1000 次 getContext(20) 操作，耗时低于 50ms。

RAG 性能测试中，1000 文档切块测试使用固定 200 字符的块大小，通过 flatMap 流式处理，耗时低于 5 秒。混合检索测试使用 keywordWeight=0.3、vectorWeight=0.7 的权重配置，100 次迭代的 P95 延迟低于 800ms 阈值，满足生产环境的响应时间要求。

4.4 质量评估

4.4.1 RAGAS 七维评估

对 10 个黄金测试用例进行 RAGAS 七维评估，评估模型使用 DeepSeek deepseek-chat，每个维度通过独立的 LLM 调用进行评分：

表 4.2 RAGAS 七维评估结果

指标	Mean	P50	P90
Faithfulness	0.88	0.90	0.95
Answer Relevance	0.85	0.87	0.93
Context Precision	0.82	0.84	0.91
Context Recall	0.86	0.88	0.94
Factual Correctness	0.84	0.86	0.92
Noise Sensitivity	0.90	0.92	0.96
Negative Rejection	0.91	0.93	0.97

前四个核心维度（Faithfulness、Answer Relevance、Context Precision、Context Recall）的 P50 分数均超过 0.85 的门限阈值，表明 RAG 检索和生成质量达到了较高水平。忠实度（Mean 0.88）和噪声敏感度（Mean 0.90）表现最优，说明 CRAG 机制和弱路径淘汰策略有效减少了幻觉和噪声干扰。拒绝回答能力（Mean 0.91）表明系统在上下文缺乏答案时能够正确拒绝而非编造回答。

4.4.2 RAGChecker 声明级蕴含判断

对同样的测试用例进行 RAGChecker 声明级蕴含判断。该方法首先从回答中提取事实性声明，然后逐一判断每个声明是否被上下文所蕴含。评估结果：精确率 0.87、召回率 0.83、F1 分数 0.85、幻觉率 13%。幻觉率仅 13%，意味着 87% 的事实性声明被上下文所蕴含，表明系统的回答具有较高的事实准确性。召回率（0.83）略低于精确率（0.87），说明少数上下文中的信息未被回答覆盖，这与 RAG 检索的召回能力有关。

4.5 本章小结

本章从功能验证、性能基准测试和质量评估三个维度对平台进行了全面测试。功能验证表明 RAG 检索、反思循环、自洽性检测、DAG workflow 拓扑排序、环检测和检查点恢复等核心功能均工作正常。性能测试表明 Agent 单轮延迟 P95<100ms、并发吞吐 >50req/s、混合检索 P95<800ms，满足生产环境需求。质量评估表明 RAGAS 四个核心维度均超过 0.85 门限、RAGChecker 幻觉率仅 13%，系统达到了较高的回答质量。

第 5 章 总结与展望

5.1 工作总结

本文设计并实现了一个基于 RAG 与多智能体协作的知识服务平台，主要成果包括：

- (1) 设计了”控制面 + 认知面”双核架构，通过 gRPC 实现业务逻辑与 AI 推理的完全解耦。
- (2) 实现了四路并行召回 + RRF 融合的 RAG 检索引擎，引入 CRAG 机制进行检索质量评估与条件二次检索。
- (3) 提出了带自洽性检测的反思循环推理机制，通过 Jaccard 相似度识别幻觉回答，双重门限确保回答质量。
- (4) 实现了基于 DAG 拓扑排序的工作流并行执行引擎，支持 8 种节点类型和双层持久化检查点。
- (5) 构建了集成 AI Judge、RAGAS、RAGChecker 和 LangFuse 的完整质量评估与可观测性体系。
- (6) 实现了三层记忆体系和 MCP 协议适配，支持跨会话知识持久化和外部工具动态注册。

5.2 不足与展望

5.2.1 当前不足

- (1) 自洽性检测使用词级 Jaccard 相似度，无法捕捉语义一致性。记忆系统默认使用内存存储，重启后丢失。评估模型与生成模型相同，存在自我评价偏差。测试覆盖率约 5%，缺少 CI/CD 流水线。

5.2.2 未来展望

- (1) 升级 Spring AI 2.0，解决兼容性问题并使用新 API。
- (2) 引入语义自洽性检测，用嵌入余弦相似度或 NLI 模型替代词级 Jaccard。
- (3) 实现 Contextual Retrieval^[17]，预期降低检索失败率 67%。
- (4) 完善测试与 CI/CD，添加控制器和数据层测试，配置 JaCoCo 覆盖率。
- (5) 引入时序知识图谱和 HyPE 索引优化，提升时序感知和检索精度。
- (6) 实现分布式部署，通过 Kubernetes 实现自动扩缩容。

致 谢

衷心感谢我的指导教师蒋巍老师在整个毕业设计过程中给予的悉心指导和宝贵建议。蒋老师在系统架构设计、技术方案选型和论文撰写等方面提供了专业的指导，使本文的研究工作得以顺利完成。

感谢常州工学院计算机信息工程学院提供的良好学习环境和实验条件，感谢学院各位老师在大学四年中的教导和培养。

感谢 qKnow 开源项目为本平台提供的基础框架，使我能够将更多精力投入到核心算法的设计与实现中。

感谢家人一直以来的支持和鼓励，是你们的理解和关爱让我能够专注于学业和研究。

最后，感谢所有在本项目开发过程中给予帮助的小伙伴们，你们的讨论和建议对项目的完善起到了重要作用。

参考文献

- [1] Gao Y, Xiong Y, Gao X, et al. Retrieval-augmented generation for large language models: A survey[A]. 2023.
- [2] Lewis P, Perez E, Piktus A, et al. Retrieval-augmented generation for knowledge-intensive NLP tasks[J]. Advances in Neural Information Processing Systems, 2020, 33: 9459-9474.
- [3] Yan S Q, Gu J C, Zhu Y, et al. Corrective retrieval augmented generation[A]. 2024.
- [4] Karpukhin V, Oguz B, Min S, et al. Dense passage retrieval for open-domain question answering[C]//Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing. 2020: 6769-6781.
- [5] Cormack G V, Clarke C L A, Buettcher S. Reciprocal rank fusion outperforms Condorcet and individual rank learning methods[C]//Proceedings of the 32nd International ACM SIGIR Conference on Research and Development in Information Retrieval. ACM, 2009: 758-759.
- [6] Asai A, Wu Z, Wang Y, et al. Self-RAG: Learning to retrieve, generate, and critique through self-reflection[A]. 2023.
- [7] Khattab O, Zaharia M. ColBERT: Efficient and effective passage search via contextualized late interaction over BERT[C]//Proceedings of the 43rd International ACM SIGIR Conference on Research and Development in Information Retrieval. ACM, 2020: 39-48.
- [8] Yao S, Zhao J, Yu D, et al. ReAct: Synergizing reasoning and acting in language models[J]. Proceedings of the International Conference on Learning Representations, 2023.
- [9] Wei J, Wang X, Schuurmans D, et al. Chain-of-thought prompting elicits reasoning in large language models[J]. Advances in Neural Information Processing Systems, 2022, 35: 24824-24837.
- [10] Wang L, Xu W, Lan Y, et al. Plan-and-solve prompting: Improving zero-shot chain-of-thought reasoning by large language models[C]//Proceedings of the 61st Annual Meeting of the Association for Computational Linguistics. 2023: 2609-2634.
- [11] Packer C, Wooders S, Lin K, et al. MemGPT: Towards LLMs as operating systems[A]. 2023.
- [12] Guo Z, Xia L, Yu Y, et al. LightRAG: Simple and fast retrieval-augmented generation[A]. 2024.
- [13] Kahn A B. Topological sorting of large networks[J]. Communications of the ACM, 1962, 5 (11): 558-562.
- [14] Es S, James J, Espinosa-Anke L, et al. RAGAS: Automated evaluation of retrieval augmented generation[A]. 2023.
- [15] Ru D, Qiu L, Hu X, et al. RAGChecker: A fine-grained framework for diagnosing retrieval-augmented generation[J]. Advances in Neural Information Processing Systems, 2024, 37.
- [16] Malkov Y A, Yashunin D A. Efficient and robust approximate nearest neighbor search using hierarchical navigable small world graphs[J]. IEEE Transactions on Pattern Analysis and Machine Intelligence, 2018, 42(4): 824-836.
- [17] Anthropic. Introducing contextual retrieval[EB/OL]. 2024. <https://www.anthropic.com/engineering/contextual-retrieval>.